

# KENYA PETROLEUM MARKET OUTLOOK

## Methods and Code Report

*Every step taken, every tool used, every assumption, and the complete code*

Companion to the Kenya Petroleum Market Outlook 2026–2035 | October 2026

## 1. Purpose and how to use this report

This report documents how the Market Outlook was produced so that every number can be traced and re-run. It covers the environment, a chronological log of what was done (including what failed), the data and its quality audit, the exact method of each analysis, an assumptions register, deviations from the original project brief, and a full code appendix. The repository (data/raw, data/clean, code, figures) reproduces everything with one command: `./run_all.sh`.

## 2. Tools and environment

| Tool                       | Used for                                                                                                                                                                                                                                             |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Python 3.12, pandas, numpy | Data loading, cleaning, reshaping, all calculations                                                                                                                                                                                                  |
| scipy (optimize, brentq)   | Fitting ETS and ARIMA by numerical optimisation; root finding for break-even oil prices                                                                                                                                                              |
| matplotlib                 | All nine figures                                                                                                                                                                                                                                     |
| python-docx                | Building both Word reports from the same numbers                                                                                                                                                                                                     |
| LibreOffice, pdftoppm      | Rendering the reports to check layout                                                                                                                                                                                                                |
| Web search and page fetch  | Retrieving the EPRA Petroleum Development Plan (PDF), 2026 price reviews, stock-cover statements, oil-market news, South Lokichar facts                                                                                                              |
| Not available              | statsmodels (not installable; no network in the code sandbox). ETS, ARIMA(0,1,1) and Newey-West OLS were therefore written directly with numpy/scipy. Excel and a self-contained HTML dashboard were built instead; SQL and Streamlit were not used. |

## 3. Chronological log of work

**Step 1 – Inventory of inputs.** Three CSVs were uploaded: monthly consumption (191 lines), national average retail prices (77 lines), town-level pump prices (4 lines: three comment lines plus a header, **no data rows**). The pump-price file was recorded as empty and excluded.

**Step 2 – Earlier access attempts (before the files arrived).** The LeadAfrik API was blocked to automated access (robots rule) and the web page exposed only 48 of 189 rows; the code sandbox had no internet. No numbers were estimated from partial data. The user supplied the three CSVs instead.

**Step 3 – Structural validation.** Footer attribution lines removed; 189 rows = 7 products × 27 months confirmed; no duplicate (month, product); only March 2026 blank (7 values); 21 rows flagged provisional (Jan–Mar 2026). Prices: 75 rows = 5 series × 15 months, complete.

**Step 4 – Reasonableness check that exposed the label error.** Annual sums per column were compared with EPRA PDP 2024 actuals (diesel 2,193.6 kt; petrol 1,472.7; LPG 414.9; kerosene 37.1; jet 765.1; fuel oil 301.9). The column labelled “LPG” summed to 1,472.7 kt, equal to PDP petrol; “Other” summed to 414.9, equal to PDP LPG. A full 7×6 matrix of column-versus-product errors was computed (code 01); exact matches (0.0%) were unique per product. The relabelling was then confirmed against 2025 KNBS facts: diesel 2,419.1 kt; LPG 475.9 kt (+14.7%); diesel+petrol 71.4% of demand; total 5.7 Mt. All four independent checks held.

**Step 5 – External context gathering.** EPRA PDP 2025–2029 read in full (demand equations, scenario growth rates, base-year data, storage capacity, LPG balance). Economic Survey 2026 figures located in press coverage. The discovery that a Gulf war began on 28 Feb 2026, with fuel shortages and a price spike, changed the framing of the whole report: the dataset ends one month into the shock. Pump-price reviews (March–September 2026), EPRA landed-cost statements, stock-cover statements and South Lokichar status were searched individually.

**Step 6 – Demand analytics (code 02).** Annual totals, shares, growth, month-on-month and year-on-year changes, rolling means, coefficient of variation, calendar-month seasonal indices, log-linear trends, LPG–kerosene correlations; figures 1–3.

**Step 7 – Forecast tournament (code 03).** Eight models, rolling-origin evaluation, selection by sMAPE, 12-month forecasts with 80/95% intervals; figure 4.

**Step 8 – Prices, elasticity and pass-through (code 04).** Price statistics; own log-log regression with Newey–West errors (result: wrong sign, rejected); elasticities derived from the PDP equations; petrol pass-through from EPRA landed costs; figure 5.

**Step 9 – Scenarios, supply security and risk (code 05).** Three scenarios to 2035; PDP back-test; stock-cover gaps; capacity versus requirement; LPG storage by the PDP’s own method; risk scores with weight sensitivity; figures 6–7.

**Step 10 – Project economics (code 06).** Discounted cash-flow model for South Lokichar with two fiscal cases, break-evens, price×output grid and tornado; figures 8–9.

**Step 11 – Extended analysis (code 09).** Product profiles; regional price spreads from press-reported EPRA maxima; price indices; cooking-fuel cost per useful MJ (illustrative efficiencies); landed-cost import-bill arithmetic; inventory value and carrying cost of 15/30/60/90 days of cover; petrol-displacement arithmetic; one-at-a-time scenario sensitivities; 2,000-draw Monte Carlo on South Lokichar; additional figures 10–13.

**Step 12 – Excel model (code 10).** Workbook with live formulas (scenarios, supply security, LPG storage, project cash flow with cost-recovery pool, IRR/NPV) and static sensitivity tables. Recalculated in LibreOffice and compared with Python: scenario totals identical (6,305.7 kt in 2030; 7,157.3 kt in 2035); project NPV10 1,550.5 vs 1,551; government NPV 2,812.5 vs 2,813; contractor NPV –1,262 vs –1,260. A first version differed (1,370) because the Excel plateau ended in 2036 while the Python model runs to 2037; the Excel model was corrected and re-verified.

**Step 13 – Report assembly and verification (codes 07, 08, 11).** Reports generated from the CSV outputs; numbers in text cross-checked against the CSVs; full clean re-run of the pipeline (exit status 0) to confirm reproducibility; layout checked by rendering to PDF.

## 4. Data audit and reconciliation

| Product (corrected) | 2024 sum (kt) | 2025 sum (kt) | Reference 2024 (PDP) | Reference 2025 (KNBS) | Error 2024 |
|---------------------|---------------|---------------|----------------------|-----------------------|------------|
| Diesel (AGO)        | 2,193.6       | 2,419.1       | 2,193.6              | 2,419.1               | +0.0%      |
| Petrol (PMS)        | 1,472.7       | 1,657.5       | 1,472.7              |                       | +0.0%      |
| LPG                 | 414.9         | 476.0         | 414.9                | 475.9                 | +0.0%      |
| Jet fuel*           | 732.3         | 701.3         | 765.1                |                       | -4.3%      |
| Fuel oil/other*     | 250.1         | 413.3         | 301.9                |                       | -17.2%     |
| Kerosene (IK)       | 37.1          | 44.1          | 37.1                 |                       | +0.0%      |
| Avgas               | 1.1           | 1.2           |                      |                       |            |
| Total               | 5,101.8       | 5,712.5       |                      | 5,700.0               |            |

**Relabelling map applied (code/common.py):** “LPG” → Petrol (PMS); “Other” → LPG; “Motor Spirit” → Jet fuel\* (probable); “Jet Oil” → Fuel oil/other\* (probable); AGO, kerosene and avgas unchanged. Evidence strength: four exact matches; the jet and fuel-oil assignments rest on magnitude (–4.3% and –17.2% from the PDP annual figures). These differences are consistent with LEI versus annual-survey definitions, but this has not been confirmed.

## 5. Methods in detail

### 5.1 Demand analytics

- Growth:  $YoY = x_t / x_{t-12} - 1$ ;  $MoM = x_t / x_{t-1} - 1$ . Rolling means over 3 and 12 months (12-month undefined before month 12).
- Volatility: coefficient of variation of monthly volumes over 2024–25 (std / mean).
- Seasonality: ratio of each month to its calendar-year mean, averaged over 2024 and 2025 (two observations per month).
- Trend: slope of  $\ln(\text{volume})$  on time; annualised as  $\exp(12 \times \text{slope}) - 1$ .
- Unit conversion: million litres per kt from the PDP base year (diesel 1.189, petrol 1.388, jet 1.271).

### 5.2 Forecast tournament

| Model                          | Definition                                                                                                              |
|--------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| Naive                          | Last observation carried forward                                                                                        |
| Mean (last 6m)                 | Mean of the latest six months                                                                                           |
| Drift                          | Last value + average historical change                                                                                  |
| Seasonal naive                 | Value 12 months earlier                                                                                                 |
| Seasonal naive $\times$ growth | Seasonal naive scaled by (sum of latest 6 months / sum of the same 6 months a year earlier)                             |
| Damped-trend ETS               | Additive level + damped trend, smoothing parameters and damping fitted by minimising squared one-step errors (L-BFGS-B) |
| ARIMA(0,1,1)                   | MA parameter fitted by conditional sum of squares; forecast flat from the one-step forecast                             |
| Log-trend + seasonal (shrunk)  | Log-linear trend plus month-of-year residual means, shrunk 50%                                                          |

**Evaluation.** Rolling-origin: for each origin  $o$  from 18 to 25 (training on the first  $o$  months), forecast up to 6 months ahead (fewer near the end of the sample). This produces 33 forecast errors per series and model. Metrics: MAE, RMSE, MAPE, sMAPE =  $\text{mean}(2|e| / (|a| + |f|))$ , MASE (MAE scaled by the in-sample seasonal-naive MAE). Selection criterion: lowest sMAPE per series. **Intervals:** forecast  $\pm z \times \text{RMSE}$  of the selected model's errors at that horizon ( $z = 1.2816$  for 80%, 1.96 for 95%); for horizons beyond 6, RMSE at horizon  $6 \times \sqrt{h/6}$ . **Why not SARIMA/ARIMAX/Holt-Winters:** two seasonal cycles are too few to estimate seasonal states reliably and no long monthly exogenous series (GDP, FX, Brent) was available.

### 5.3 Elasticities and pass-through

Own estimate:  $\ln Q_t = \beta_0 + \beta_1 \ln P_t + \beta_2 t + \varepsilon_t$ , 14 monthly observations (Jan 2025–Feb 2026), Newey-West (lag 2) standard errors. Result:  $\beta_1 = +1.65$  (diesel) and  $+0.69$  (petrol), wrong sign, so it was rejected. PDP-implied elasticities: short-run  $\varepsilon = \text{coefficient} \times P / Q$  at base-year values; long-run  $\varepsilon = \varepsilon_{\text{SR}} / (1 - \phi)$ , where  $\phi$  is the lagged-dependent-variable coefficient (0.6754 diesel; 0.6284 petrol). Pass-through:  $\Delta(\text{pump price}) / \Delta(\text{landed cost})$ , landed cost converted at an assumed exchange rate, with and without 16% VAT.

### 5.4 Scenarios

For product  $p$ , scenario  $s$  and year  $t$ :  $Q_t = Q_{t-1} \times \exp[\ln(1 + g_{\{p,s,t\}}) + \text{adj}_t]$ , where  $g$  is the PDP growth rate for 2025–29 and  $0.8 \times$  that rate for 2030–35, and  $\text{adj}$  is the price-shock term: in 2026,  $\varepsilon_{\text{SR}} \times \ln(1 + \Delta P)$ ; in 2027 and 2028,  $(\varepsilon_{\text{LR}} - \varepsilon_{\text{SR}}) \times \ln(1 + \Delta P) / 2$  each. Assumed price shocks and elasticities are listed in the register below.

### 5.5 Supply security

Days of cover = stock / average daily demand, with daily demand = 2025 annual demand (kt)  $\times$  litres per kt / 365. Stock required for 30 days =  $30 \times$  daily demand. LPG storage: required = demand  $\times (1 + 30/365) / 12$ , which reproduces the PDP's 2025 figures (482.86 kt planned supply; spare ullage about 4 kt).

## 5.6 Project economics

Annual model, 2026–2060 (stops when production ends). Revenue = barrels × (Brent – differential). Royalty on gross revenue. Cost recovery: opex and capex accumulate in a pool recovered each year up to a cap share of revenue net of royalty. Profit oil = net revenue – cost recovery, split with the government. Government take = royalty + government profit oil share. Contractor cash flow = revenue – royalty – government profit oil – opex – capex. Project (pre-fiscal) cash flow = revenue – opex – capex. NPV discounted at 10% to 2026; IRR by root finding; break-even Brent by root finding of NPV = 0. Tornado: each driver moved one at a time.

## 5.7 Extended analysis

- Import-bill arithmetic: volume (ML) = 2026 baseline kt × ML per kt; bill (US\$ m) = ML × landed cost (US\$/m<sup>3</sup>) / 1,000; KSh at 129.2. Constant volumes; illustrative.
- Inventory cost: days × average daily demand (2025) × landed cost; carrying cost = 10% of inventory value (financing only).
- Cooking-fuel cost per useful MJ = (price per kg) / (energy content × stove efficiency); energy content 46 MJ/kg LPG, 29 MJ/kg charcoal; efficiencies are illustrative.
- Petrol displacement: share × 2035 baseline petrol kt × ML/kt × Aug-2026 landed cost.
- Scenario sensitivities: the scenario function is re-run with one input changed (fade 0.6/1.0; shock 0/1.5×; elasticities 0.5×/2×; underlying growth +2/–1 points).
- Regional spread = town price – Mombasa price, in KSh and %.

## 5.8 Monte Carlo (South Lokichar)

2,000 draws (seed 42) from independent distributions: Brent log-normal(ln 70, 0.22) clipped to [35, 140]; differential U(3, 10); plateau triangular(40, 60, 100); capex triangular(5,000, 6,000, 8,500); opex + tariff triangular(15, 21, 28), split 12:9 between opex and tariff. Each draw runs the full cash-flow model. Reported: P10, median, P90, mean, probability NPV < 0, and Pearson correlation of each input with project NPV. Independence of inputs is a simplification (in reality capex and oil price co-move).

## 5.9 Excel model structure and verification

| Sheet             | Content                                                                                               | Live?  |
|-------------------|-------------------------------------------------------------------------------------------------------|--------|
| README            | Instructions and caveats                                                                              | —      |
| Dashboard         | Key outputs linked to model sheets                                                                    | Yes    |
| Assumptions       | Scenario selector (C3), growth, price shocks, elasticities, FX, conversion factors                    | Inputs |
| Demand_Data       | Relabelled monthly data; annual SUMIFS, growth, shares                                                | Yes    |
| Prices            | National price series; change and CV                                                                  | Yes    |
| Scenarios         | 2025–2035 by product, driven by selector                                                              | Yes    |
| Supply_Security   | Daily demand, capacity days, 30-day need, shortfall vs reported cover, traffic light, inventory value | Yes    |
| LPG_Storage       | Required storage vs installed + new capacity                                                          | Yes    |
| Project_Economics | 35-year cash flow, cost-recovery pool, NPV, IRR, government and contractor results                    | Yes    |
| Sensitivity       | Static tables from Python (grids, tornado, fiscal cases, scenario sensitivities)                      | Static |

Verification: the workbook was recalculated headlessly in LibreOffice and no cell returned an error; scenario and project outputs match the Python results to the figures quoted in Step 12.

## 6. Assumptions register

| Assumption                              | Value                                                                                                                                                                         | Basis                                                  |
|-----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------|
| Litres per tonne                        | Diesel 1.189; petrol 1.388; jet 1.271; kerosene 1.271 ML/kt                                                                                                                   | Implied by PDP 2024 base                               |
| Growth 2025–29                          | PDP baseline / optimistic / pessimistic by product                                                                                                                            | EPRA PDP, Section 4                                    |
| Growth 2030–35                          | 0.8 × the 2025–29 rate                                                                                                                                                        | Author assumption                                      |
| 2026 average price rise (high/base/low) | Diesel 10/20/30%; petrol 8/15/22%; LPG 5/10/20%; kerosene 10/20/30%; jet 0                                                                                                    | Author, informed by Mar–Sep 2026 EPRA reviews          |
| LPG elasticity (SR/LR)                  | –0.15 / –0.30                                                                                                                                                                 | Author assumption                                      |
| Kerosene elasticity (SR/LR)             | –0.2 / –0.5                                                                                                                                                                   | Author assumption                                      |
| Jet elasticity                          | 0                                                                                                                                                                             | Author assumption                                      |
| Fuel oil path                           | PDP: +4.68%/yr to 2027, then –3.19%/yr                                                                                                                                        | EPRA PDP                                               |
| USD/KES for pass-through                | 129.2 (flat); 125–135 tested                                                                                                                                                  | Author assumption                                      |
| Stock-cover standard                    | 30 days (15 operational + 15 strategic)                                                                                                                                       | EPRA PDP                                               |
| Traffic-light thresholds                | Green ≥ 30; amber 15–29; red < 15 days                                                                                                                                        | Author framework (not official)                        |
| Risk weights                            | Stock 25%, price 20%, infrastructure 15%, volatility 15%, concentration 15%, forecast 10%                                                                                     | Author framework; equal and stock-heavy weights tested |
| Monte Carlo distributions               | See Section 5.8                                                                                                                                                               | Author assumptions                                     |
| Cooking-fuel efficiencies               | LPG 45–55%; charcoal 20–30%                                                                                                                                                   | Author illustration                                    |
| Inventory financing rate                | 10%                                                                                                                                                                           | Author assumption                                      |
| South Lokichar                          | Brent \$70; differential \$6; plateau 60 kbpd; capex \$6.0 bn; opex \$12 + tariff \$9; 10% discount; royalty 10%; cost cap 60%/80%; govt profit oil 50%/35%; no corporate tax | Headline facts from press; all else author assumptions |

## 7. Deviations from the original project brief

| Original brief item                                        | What was done                                                                                            | Why                                                   |
|------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|-------------------------------------------------------|
| Use LeadAfrik column labels as published                   | Relabelled petrol/LPG/jet/fuel oil columns                                                               | Labels failed reconciliation with EPRA and KNBS       |
| SARIMA, ARIMAX, Holt-Winters                               | ETS (damped), ARIMA(0,1,1), growth-adjusted seasonal naïve, shrunk-seasonal trend                        | 26 observations; no exogenous series                  |
| Forecast to 2035 with intervals                            | 12-month statistical forecast with intervals; 2025–35 scenario projections without statistical intervals | Horizon limited by data                               |
| Elasticity from own regression                             | PDP-implied elasticities                                                                                 | Own regression wrong-signed                           |
| Brent / FX pass-through regression                         | Two-point landed-cost pass-through                                                                       | No monthly Brent/FX data                              |
| Import dependence ratios                                   | Structural 100% dependence; flagged data gap in imports vs demand                                        | Imports are all refined; Survey tables not accessed   |
| Stock-cover days by product (incl. LPG)                    | Petrol, diesel, jet from statements; LPG via storage headroom                                            | No LPG stock cover published                          |
| Diesel storage 224.5 → 254.7 ML (from an earlier document) | Not used                                                                                                 | Could not be verified in the PDP text retrieved       |
| South Lokichar Phase 1 20 kbpd / Phase 2 50 kbpd           | Not used; 60–100 kbpd press range used                                                                   | Conflict with press reports, unverified               |
| Excel model                                                | Built (live formulas; verified against Python)                                                           | Delivered in the second iteration                     |
| Interactive HTML dashboard (7 tabs)                        | Delivered                                                                                                | Hand-built with embedded data; open it in any browser |

| Original brief item       | What was done                                                        | Why                                                                                                    |
|---------------------------|----------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| Streamlit app, SQL schema | Not built                                                            | Not feasible to test in this environment; outputs are CSVs and the Excel model is the interactive tool |
| Regional price spreads    | Done from press-reported EPRA maxima (May and September 2026 cycles) | Town-level file was empty; secondary source                                                            |

## 8. Validation and checks performed

- Row counts and key uniqueness of the raw files.
- Reconciliation of corrected columns to PDP 2024 and KNBS 2025 (exact on four products; total within 0.2% of KNBS 2025).
- Hold-out forecasting error reported on rolling origins; the PDP's own 2025 baseline back-tested against actual 2025 (5.3% under).
- Recomputation of PDP LPG storage logic (2025 spare ullage  $\approx$  4 kt) before applying it to new demand.
- Cross-checking of every figure in the narrative against CSV outputs; consistency check of PDP supply (6,420) minus demand (5,933) = 30 days of demand.
- Full clean re-run from raw data to final reports and the Excel model.
- Second-pass review of every new number in the expanded report against the CSV outputs; this corrected four statements (petrol demand response in the stress test, KSh cost of holding prices flat, diesel first-year demand loss, ARIMA metric range).

## 9. Reproducing the work

```
$ cd project_root
$ pip install pandas numpy scipy matplotlib python-docx
$ ./run_all.sh
# outputs: data/clean/*.csv, figures/*.png, out/Kenya_Petroleum_Market_Outlook_2026-2035.docx
```

# Appendix A. Code listings

## A.1 code/common.py – Shared labels, conversion factors, loaders

```
import pandas as pd, numpy as np
MONTHS=['January','February','March','April','May','June','July','August','September','October','November','December']
# Relabelling established in 01_prepare_data.py (exact reconciliation to EPRA PDP 2024 actuals / KNBS 2025)
RELABEL={'AGO (Light Diesel Oil)': 'Diesel (AGO)', 'LPG': 'Petrol (PMS)', 'Other': 'LPG',
         'Illuminating Kerosene': 'Kerosene (IK)', 'Aviation Gasoline': 'Avgas',
         'Motor Spirit': 'Jet fuel*', 'Jet Oil': 'Fuel oil/other*'}
PRODUCTS=['Diesel (AGO)', 'Petrol (PMS)', 'LPG', 'Jet fuel*', 'Fuel oil/other*', 'Kerosene (IK)', 'Avgas']
# kt -> million litres: factors implied by EPRA PDP 2025-29 (million litres / kt, 2024 base)
ML_PER_KT={'Diesel (AGO)': 2608.20/2193.6, 'Petrol (PMS)': 2044.11/1472.7, 'Jet fuel*': 972.49/765.1, 'Kerosene (IK)': 47.17/37.1}
def load_cons():
    c=pd.read_csv('data/raw/leadafrik-consumption-of-petroleum-fuels-kenya-monthly.csv').dropna(subset=['month'])
    c['date']=pd.to_datetime(dict(year=c.year.astype(int), month=c.month.map({m:i+1 for i,m in enumerate(MONTHS)}), day=1))
    c['product']=c.fuel_type.map(RELABEL); c['kt']=c.consumption_volume
    w=c.pivot(index='date', columns='product', values='kt')[PRODUCTS]
    w['Total']=w[PRODUCTS].sum(axis=1, min_count=7)
    return w
def load_prices():
    p=pd.read_csv('data/raw/leadafrik-national-average-retail-prices-for-selected-fuels-in-kenya.csv').dropna(subset=['month'])
    p['date']=pd.to_datetime(dict(year=p.year.astype(int), month=p.month.map({m:i+1 for i,m in enumerate(MONTHS)}), day=1))
    return p.pivot(index='date', columns='fuel_type', values='average_retail_price')
```

## A.2 code/01\_prepare\_data.py – Validation and reconciliation

```
"""01 - Load, validate, and reconcile LeadAfrik consumption data against independent sources."""
import pandas as pd, numpy as np
RAW='data/raw/'
MONTHS=['January','February','March','April','May','June','July','August','September','October','November','December']
c=pd.read_csv(RAW+'leadafrik-consumption-of-petroleum-fuels-kenya-monthly.csv')
c=c.dropna(subset=['month']).copy()
c['year']=c.year.astype(int); c['m']=c.month.map({m:i+1 for i,m in enumerate(MONTHS)})
c['date']=pd.to_datetime(dict(year=c.year, month=c.m, day=1))
c['provisional']=c.provisional.eq('Yes')
c=c.rename(columns={'fuel_type': 'label_as_published', 'consumption_volume': 'kt'})
assert len(c)==189 and c.groupby('label_as_published').size().eq(27).all()
print('duplicates', c.duplicated(['date', 'label_as_published']).sum(), 'missing', c.kt.isna().sum(),
      c[c.kt.isna()].date.dt.strftime('%Y-%m').unique())
w=c.pivot(index='date', columns='label_as_published', values='kt')
s24=w.loc['2024'].sum(); s25=w.loc['2025'].sum()
print(pd.DataFrame({'2024': s24, '2025': s25}).round(1), s24.sum(), s25.sum())
pdp24={'AGO': 2193.6, 'PMS': 1472.7, 'LPG': 414.9, 'IK': 37.1, 'JET': 765.1, 'FO': 301.9}
M=pd.DataFrame({p: {col: (s24[col]-v)/v*100 for col in w.columns} for p,v in pdp24.items()}).round(1)
print(M)
# ---- save analysis-ready relabelled dataset and reconciliation table ----
import sys; sys.path.insert(0, 'code')
from common import load_cons
W=load_cons(); W.round(3).to_csv('data/clean/consumption_relabelled_kt.csv')
rec=pd.DataFrame({'2024_kt': W.loc['2024'].sum(), '2025_kt': W.loc['2025'].sum()}).round(1)
rec['reference_2024']=pd.Series({'Diesel (AGO)': 2193.6, 'Petrol (PMS)': 1472.7, 'LPG': 414.9, 'Kerosene (IK)': 37.1, 'Jet fuel*': 765.1, 'Fuel oil/other*': 301.9})
rec['reference_2025_KNBS']=pd.Series({'Diesel (AGO)': 2419.1, 'LPG': 475.9, 'Total': 5700.0})
rec['err_2024_%']=((rec['2024_kt']/rec.reference_2024-1)*100).round(1); rec.to_csv('data/clean/reconciliation.csv');
print(rec)
print('diesel+petrol share 2025 %:', round((W.loc['2025', 'Diesel (AGO)'].sum()+W.loc['2025', 'Petrol (PMS)'].sum())/W.loc['2025', 'Total'].sum()*100, 1))
```

## A.3 code/02\_demand\_analysis.py – Demand analytics and figures 1–3

```
import pandas as pd, numpy as np, matplotlib; matplotlib.use('Agg'); import matplotlib.pyplot as plt
from common import *
w=load_cons(); P=PRODUCTS[:6]; W=w.dropna() # drop Mar-2026 (blank)
out={}
# annual, shares, growth
ann=w.loc['2024':'2025'].groupby(lambda d:d.year).sum()
ann.loc['growth%']=(ann.loc[2025]/ann.loc[2024]-1)*100
share=(ann.loc[[2024,2025]].div(ann.loc[[2024,2025], 'Total'], axis=0)*100)
print(ann.round(1).T); print(share.round(1).T)
# Jan-Feb YTD growth 2026 vs 2025
ytd=(W.loc['2026-01':'2026-02'].sum().values/W.loc['2025-01':'2025-02'].sum().values-1)*100
ytd=pd.Series(ytd, index=W.columns); print('YTD26 growth', ytd.round(1).to_dict())
# YoY monthly, MoM, rolling, CV, seasonality
yoy=W.pct_change(12)*100; mom=W.pct_change()*100
r3=W.rolling(3).mean(); r12=W.rolling(12).mean()
cv=(W.loc['2024':'2025'].std()/W.loc['2024':'2025'].mean()*100)
# detrended seasonal index: ratio to centred 12-month mean not feasible (n=26) -> use ratio to calendar-year mean, avg over 2024,2025
si=pd.concat([W.loc[str(y)].div(W.loc[str(y)].mean()) for y in (2024,2025)]); si=si.groupby(si.index.month).mean()
print('CV%', cv.round(1).to_dict()); print(si[['Diesel (AGO)', 'Petrol (PMS)', 'LPG', 'Jet fuel*', 'Total']].round(3))
# trend growth: log-linear trend per product (annualised)
t=np.arange(len(W)); tr={}
for p in W.columns:
    b=np.polyfit(t, np.log(W[p].values), 1)[0]; tr[p]=(np.exp(12*b)-1)*100
print('trend ann%', {k: round(v, 1) for k,v in tr.items()})
```

```

pd.DataFrame({'ann2024':ann.loc[2024],'ann2025':ann.loc[2025],'growth%':ann.loc['growth
%'],'share24':share.loc[2024],'share25':share.loc[2025],'ytd26%':ytd,'CV%':cv,'trend
%':pd.Series(tr)).round(2).to_csv('data/clean/demand_summary.csv')
yoy.round(2).to_csv('data/clean/yoy.csv'); si.round(4).to_csv('data/clean/seasonal_index.csv')
# correlation LPG vs kerosene, petrol vs diesel (levels and yoy)
print('corr LPG-IK levels',W['LPG'].corr(W['Kerosene (IK)']).round(2),'yoy',yoy['LPG'].corr(yoy['Kerosene
(IK)'])).round(2))
# figures
plt.rcParams.update({'font.size':9,'axes.spines.top':False,'axes.spines.right':False})
fig,ax=plt.subplots(figsize=(7.5,3.6))
for p in ['Diesel (AG0)','Petrol (PMS)','LPG','Jet fuel*']: ax.plot(W.index,W[p],marker='o',ms=3,label=p)
ax.set_ylabel('Thousand tonnes / month'); ax.legend(ncol=4,frameon=False,loc='upper center',bbox_to_anchor=(.5,1.15));
ax.grid(alpha=.25)
fig.tight_layout(); fig.savefig('figures/f1_monthly_demand.png',dpi=200); plt.close()
fig,ax=plt.subplots(figsize=(7.5,3.4)); x=np.arange(len(P));
ax.bar(x-.2,ann.loc[2024,P],.4,label='2024'); ax.bar(x+.2,ann.loc[2025,P],.4,label='2025')
for i,p in enumerate(P): ax.text(i+.2,ann.loc[2025,p]+30,f"{ann.loc['growth%']:{p}:+.0f}%",ha='center',fontsize=8)
ax.set_xticks(x); ax.set_xticklabels(P,rotation=15); ax.set_ylabel('kt / year'); ax.legend(frameon=False);
fig.tight_layout(); fig.savefig('figures/f2_annual.png',dpi=200); plt.close()
fig,ax=plt.subplots(figsize=(7.5,3.2));
for p in ['Diesel (AG0)','Petrol (PMS)','LPG','Total']: ax.plot(yoy.index,yoy[p],label=p)
ax.axhline(0,c='k',lw=.6); ax.set_ylabel('YoY growth, %'); ax.legend(ncol=4,frameon=False); ax.grid(alpha=.25);
fig.tight_layout(); fig.savefig('figures/f3_yoy.png',dpi=200); plt.close()

```

## A.4 code/03\_forecasting.py – Forecast tournament and 12-month forecasts

```

"""03 - Forecasting tournament (rolling-origin) on 26 monthly observations. No statsmodels available: models
implemented with numpy/scipy."""
import numpy as np, pandas as pd, matplotlib; matplotlib.use('Agg'); import matplotlib.pyplot as plt
from scipy.optimize import minimize
from common import *
w=load_cons().dropna(); SERIES=['Diesel (AG0)','Petrol (PMS)','LPG','Jet fuel*','Total']
def f_naive(y,h): return np.repeat(y[-1],h)
def f_mean6(y,h): return np.repeat(y[-6:].mean(),h)
def f_drift(y,h):
    d=(y[-1]-y[0])/(len(y)-1); return y[-1]+d*np.arange(1,h+1)
def f_snaive(y,h): return np.array([y[-12+((i)%12)] for i in range(h)])
def f_snaive_g(y,h):
    # seasonal naive scaled by YoY growth of the latest 6 months vs same months a year earlier
    g=y[-6:].sum()/y[-18:-12].sum(); return f_snaive(y,h)*g
def f_ets(y,h): # damped additive trend, SSE-fit
    def run(p):
        a,b,ph=p; l,tr=y[0],y[1]-y[0]; sse=0
        for v in y[1:]:
            f=l+ph*tr; e=v-f; sse+=e*e; ln=f+a*e; tr=ph*tr+b*(ln-l-ph*tr)*1.0 if False else ph*tr+a*b*e; l=ln
        return sse,l,tr
    r=minimize(lambda p:run(p)[0],[.3,.1,.9],bounds=[(.01,1),(0.01,1),(0.8,.98)],method='L-BFGS-B')
    _,l,tr=run(r.x); ph=r.x[2]; return np.array([l+sum(ph**j for j in range(1,k+1))*tr for k in range(1,h+1)])
def f_arima011(y,h): # ARIMA(0,1,1) with no drift, CSS
    d=np.diff(y)
    def css(th):
        e=0;s=0
        for x in d: e=x-th[0]*e; s+=e*e
        return s
    th=minimize(css,[-.3],bounds=[(-.95,.95)],method='L-BFGS-B').x[0]
    e=0
    for x in d: e=x-th*e
    f=np.empty(h); f[0]=y[-1]+th*e; f[1:]=f[0]; return f
def f_trend_seas(y,h): # log-linear trend + month-of-year ratio (needs >=12 obs; ratios from available months)
    n=len(y); t=np.arange(n); b=np.polyfit(t,np.log(y),1); res=np.log(y)-np.polyval(b,t)
    sidx=np.array([res[i:12].mean() if len(res[i:12]) else 0 for i in range(12)]); sidx=sidx.mean()
    # month position: origin index i%12 matches calendar position because series starts in January
    return np.array([np.exp(np.polyval(b,n+k)+sidx[(n+k)%12]*0.5) for k in range(h)]) # seasonal effect shrunk 50%
MODELS={'Naive':f_naive,'Mean (last 6m)':f_mean6,'Drift':f_drift,'Seasonal naive':f_snaive,'Seasonal naive x
growth':f_snaive_g,
        'Damped-trend ETS':f_ets,'ARIMA(0,1,1)':f_arima011,'Log-trend + seasonal (shrunk)':f_trend_seas}
def smape(a,f): return np.mean(2*np.abs(a-f)/(np.abs(a)+np.abs(f)))*100
H=6; rows=[]; errs={}
for s in SERIES:
    y=w[s].values
    for m,fn in MODELS.items():
        for o in range(18,len(y)): # origin: train on y[:o]; need >=18 so growth-adjusted snaive is defined
            hh=min(H,len(y)-o); f=fn(y[:o],hh); a=y[o:ohh]
            for k in range(hh): rows.append((s,m,o,k+1,a[k],f[k]))
R=pd.DataFrame(rows,columns=['series','model','origin','h','actual','fc'])
R['ape']=np.abs(R.e)/R.actual*100; R['sape']=2*np.abs(R.e)/(R.actual+R.fc)*100
sc=R.groupby(['series','model']).agg(n=('e','size'),MAE=('e',lambda x:np.abs(x).mean()),RMSE=('e',lambda
x:np.sqrt((x**2).mean()))),MAPE=('ape','mean'),sMAPE=('sape','mean')).reset_index()
# MASE vs in-sample seasonal-naive scale
for s in SERIES:
    y=w[s].values; sc=np.mean(np.abs(y[12:]-y[:12])); sc.loc[sc.series==s,'MASE']=sc.loc[sc.series==s,'MAE']/sc_
sc=sc.round(3); sc.to_csv('data/clean/forecast_tournament.csv',index=False)
best=sc.loc[sc.groupby('series').sMAPE.idxmin()][['series','model','MAPE','sMAPE','RMSE','MASE']];
print(best.to_string())
print(sc.pivot(index='model',columns='series',values='sMAPE').round(2))
# ----- final forecasts, Mar-2026 .. Feb-2027, 80/95% intervals from rolling-origin errors of the selected model -----
horizon=12; out=[]
fut=pd.date_range('2026-03-01',periods=horizon,freq='MS')
for s in SERIES:
    mname=best.set_index('series').loc[s,'model']; fn=MODELS[mname]; y=w[s].values
    fc=fn(y,horizon)
    e=R[(R.series==s)&(R.model==mname)]; sig={k:np.sqrt((e[e.h==k].e**2).mean()) for k in range(1,H+1)}
    for k in range(1,horizon+1):
        sg=sig[k] if k<=H else sig[H]*np.sqrt(k/H)
        out.append((s,mname,fut[k-1],fc[k-1],fc[k-1]-1.2816*sg,fc[k-1]+1.2816*sg,fc[k-1]-1.96*sg,fc[k-1]+1.96*sg))

```

```

F=pd.DataFrame(out,columns=['series','model','date','fc','lo80','hi80','lo95','hi95']);
F.round(1).to_csv('data/clean/forecast_12m.csv',index=False)
print([F[F.series.isin(['Diesel (AG0)','Total'])].round(1).to_string()])
# annualised Mar26-Feb27
print(F.groupby('series')[['fc','lo80','hi80','lo95','hi95']].sum().round(0))
# fig
fig,axs=plt.subplots(2,2,figsize=(7.8,5.4));
for ax,s in zip(axs.ravel(),['Diesel (AG0)','Petrol (PMS)','LPG','Total']):
    f=F[F.series==s]; ax.plot(w.index,w[s],o='ms',c='#1f4e79'); ax.plot(f.date,f.fc,c='#c0392b');
    ax.fill_between(f.date,f.lo95,f.hi95,color='#c0392b',alpha=.12);
ax.fill_between(f.date,f.lo80,f.hi80,color='#c0392b',alpha=.22)
    ax.set_title(f'{s} [{f.model.iloc[0]}]',fontsize=8.5); ax.grid(alpha=.25); ax.tick_params(labelsize=7)
fig.autofmt_xdate(); fig.tight_layout(); fig.savefig('figures/f4_forecasts.png',dpi=200); plt.close()
R.round(3).to_csv('data/clean/forecast_rolling_errors.csv',index=False)

```

## A.5 code/04\_prices\_elasticity.py – Prices, elasticity, pass-through

```

"""04 - Price analytics, elasticity (own estimate + PDP-implied) and landed-cost pass-through."""
import numpy as np, pandas as pd, matplotlib; matplotlib.use('Agg'); import matplotlib.pyplot as plt
from common import *
w=load_cons(); p=load_prices(); print(p.round(2).to_string())
pr=p.rename(columns={'Motor Gasoline Premium':'petrol','Light Diesel Oil':'diesel','Illuminating
Kerosene':'kerosene','L.P.G':'lpg13','Charcoal':'charcoal'})
pr['lpg_kg']=pr.lpg13/13; pr['lpg_charcoal_ratio']=pr.lpg_kg/pr.charcoal
stats=pd.DataFrame({'min':pr.min(), 'max':pr.max(), 'mean':pr.mean(), 'cv%':pr.std()/pr.mean()*100, 'chg_%_Jan25_Mar26':
(pr.iloc[-1]/pr.iloc[0]-1)*100}).round(2); print(stats); stats.to_csv('data/clean/price_stats.csv')
print('distinct petrol/diesel price levels:',pr.petrol.nunique(),pr.diesel.nunique())
# ---- own log-log estimate, OLS with Newey-West(2) SE ----
def ols(y,X,lag=2):
    X=np.column_stack([np.ones(len(y)),X]); b=np.linalg.lstsq(X,y,rcond=None)[0]; e=y-X@b; n,k=X.shape
    S=(X*e[:,None]).T@X*e[:,None]
    for L in range(1,lag+1):
        G=(X[L:]*e[L:,:None]).T@X[L:]*e[L:,:None]; S+=(1-L/(lag+1))*(G+G.T)
    XtXi=np.linalg.inv(X.T@X); V=XtXi@S@XtXi*n/(n-k); se=np.sqrt(np.diag(V)); return b,se,e
res=[]
for prod,col in [('Diesel (AG0)','diesel'),('Petrol (PMS)','petrol')]:
    d=pd.concat([np.log(w[prod]).rename('q'),np.log(pr[col]).rename('p')],axis=1).dropna()
    t=np.arange(len(d)); m=np.array([(i+1) in () for i in range(len(d))])
    b,se,e=ols(d,q.values,np.column_stack([d.p.values,t]))
    res.append((prod,len(d),b[1],se[1],b[1]-1.96*se[1],b[1]+1.96*se[1],b[2]*12*100))
    print(prod,len(d),'price elasticity',round(b[1],2),'NW se',round(se[1],2))
E=pd.DataFrame(res,columns=['product','n','elasticity','se_NW','lo95','hi95','trend_%pa']).round(2);
E.to_csv('data/clean/own_elasticity.csv',index=False); print(E)
# ---- PDP-implied elasticities (EPRA PDP 2025-29, Section 3.4; base-year 2024 values) ----
pdp=pd.DataFrame([
    ['Petrol (PMS)',-1.0243,191.8,2044.11,0.6284],
    ['Diesel (AG0)',-2.7057,178.30,2608.20,0.6754]],columns=['product','coef_ML_per_KSh','price24','q24_ML','lag_coef'])
pdp['SR_elasticity']=pdp.coef_ML_per_KSh*pdp.price24/pdp.q24_ML; pdp['LR_elasticity']=pdp.SR_elasticity/(1-
pdp.lag_coef)
print(pdp.round(3)); pdp.round(3).to_csv('data/clean/pdp_implied_elasticity.csv',index=False)
# ---- landed cost -> pump pass-through (petrol), Nairobi max pump price vs EPRA landed cost (USD/m3) ----
# EPRA review text: Feb-2026 landed 582.11 (price cycle 15 Mar-14 Apr: Nairobi petrol 178.28, diesel 166.54);
# Aug-2026 landed petrol 874.26 / diesel 957.05 (cycle 15 Sep-14 Oct: petrol 214.03, diesel 217.86)
# diesel Feb-2026 landed not retrieved -> petrol only. FX assumption 129.2 KSh/USD (flat), sensitivity 125-135.
lc0,lc1=582.11,874.26; pp0,pp1=178.28,214.03
rows=[]
for fx in (125,129.2,135):
    dl=(lc1-lc0)/1000*fx; dp=pp1-pp0
    rows.append((fx,dl,dl*1.16,dp,dp/dl,dp/(dl*1.16)))
PT=pd.DataFrame(rows,columns=['USDKES','landed_change_KSh_L','landed_change_incl_VAT','pump_change','passthrough_ex_VA
T','passthrough_vs_VAT_inclusive']).round(2); print(PT); PT.to_csv('data/clean/passthrough.csv',index=False)
# ---- Post-March 2026 pump price path (Nairobi, news reports of EPRA reviews) ----
path=pd.DataFrame([['Mar-26 cycle (15 Mar-14 Apr)',178.28,166.54,152.78],['May-26 cycle (15 May-14
Jun)',214.25,242.92,152.78],
    ['Jul-26 cycle (15 Jul-14 Aug)',214.03,222.86,191.38],['Aug-26 cycle (15 Aug-14 Sep)',214.03,217.86,191.38],['Sep-
26 cycle (15 Sep-14 Oct)',214.03,217.86,191.38]],
    columns=['cycle','petrol','diesel','kerosene']); path.to_csv('data/clean/pump_path_2026.csv',index=False)
fig,ax=plt.subplots(figsize=(7.5,3.4));
ax.plot(pr.index,pr.petrol,'o-',label='Petrol (LeadAfrik national avg)',c='#1f4e79');
ax.plot(pr.index,pr.diesel,'o-',label='Diesel',c='#c0392b')
d2=pd.to_datetime(['2026-03-15','2026-05-15','2026-07-15','2026-08-15','2026-09-15'])
ax.plot(d2,path.petrol,'s--',c='#1f4e79',alpha=.7,label='Petrol (Nairobi max, EPRA via press)');
ax.plot(d2,path.diesel,'s--',c='#c0392b',alpha=.7,label='Diesel (Nairobi max)')
ax.axvline(pd.Timestamp('2026-02-28'),c='grey',ls=':'); ax.text(pd.Timestamp('2026-03-02'),245,'Gulf war\n28 Feb
2026',fontsize=7)
import matplotlib.dates as mdates; ax.xaxis.set_major_locator(mdates.MonthLocator(interval=3));
ax.xaxis.set_major_formatter(mdates.DateFormatter('%b-%y')); ax.set_ylabel('KSh per litre');
ax.legend(frameon=False,fontsize=7,loc='upper left'); ax.grid(alpha=.25); fig.tight_layout();
fig.savefig('figures/f5_prices.png',dpi=200); plt.close()

```

## A.6 code/05\_scenarios\_security\_risk.py – Scenarios, supply security, risk

```

"""05 - Scenario projections 2025-2035, supply security (days of cover, storage), risk framework."""
import numpy as np, pandas as pd, matplotlib; matplotlib.use('Agg'); import matplotlib.pyplot as plt
from common import *
w=load_cons(); base25=w.loc['2025'].sum() # kt, actual 2025 (LeadAfrik, reconciled)
PR=['Diesel (AG0)','Petrol (PMS)','LPG','Jet fuel*','Kerosene (IK)']
# PDP 2025-29 average annual growth by scenario (baseline, optimistic, pessimistic), % - EPRA PDP 2025-2029 Section
4.2
G={'Diesel (AG0)':(3.52,4.28,2.74),'Petrol (PMS)':(3.35,4.08,2.97),'LPG':(7.26,8.82,6.34),'Jet fuel*':
(1.87,2.17,1.70),'Kerosene (IK)':(-4.57,-3.86,-5.29)}
FADE=0.8 # ASSUMPTION: 2030-35 growth = 0.8 x the 2025-29 rate (maturing motorisation, e-mobility, slower LPG
uptake)

```

```

# 2026 price shock (average pump-price rise vs 2025 average, %) by scenario [High-demand, Baseline, Low-demand]:
ASSUMPTIONS informed by Mar-Sep 2026 EPRA reviews
SHOCK={'Diesel (AGO)':(10,20,30),'Petrol (PMS)':(8,15,22),'LPG':(5,10,20),'Jet fuel':(0,0,0),'Kerosene (IK)':
(10,20,30)}
# elasticities: PDP-implied (diesel, petrol); assumption for others (LPG -0.15 SR/-0.30 LR; kerosene -0.2/-0.5; jet 0)
EL={'Diesel (AGO)':(-0.185,-0.570),'Petrol (PMS)':(-0.096,-0.259),'LPG':(-0.15,-0.30),'Jet fuel':(0,0),'Kerosene
(IK)':(-0.2,-0.5)}
SC={'High-demand':1,'Baseline':0,'Low-demand':2} # index into G (opt=1, base=0, pess=2)
SH={'High-demand':0,'Baseline':1,'Low-demand':2}
years=list(range(2025,2036)); rows=[]
for sc in SC:
    for p in PR:
        sr,lr=EL[p]; shock=np.log(1+SHOCK[p][SH[sc]]/100); q=base25[p]
        rows.append((sc,p,2025,q))
        for y in years[1:]:
            g=G[p][SC[sc]]/100*(1 if y<=2029 else FADE)
            adj=0
            if y==2026: adj=sr*shock
            elif y in (2027,2028): adj=(lr-sr)*shock/2 # remainder of long-run response phased over 2 years
            q=q*np.exp(np.log(1+g)+adj); rows.append((sc,p,y,q))
# fuel oil/other (label uncertain) follows PDP path: + to 2027 then decline; avgas held flat at 2025 level
F0={'High-demand':(4.95,-3.13),'Baseline':(4.68,-3.19),'Low-demand':(4.44,-3.26)}
for sc in SC:
    q=base25['Fuel oil/other*']; rows.append((sc,'Fuel oil/other*',2025,q))
    for y in years[1:]:
        q*=1+(F0[sc][0] if y<=2027 else F0[sc][1])/100; rows.append((sc,'Fuel oil/other*',y,q))
    for y in years: rows.append((sc,'Avgas',y,base25['Avgas']))
S=pd.DataFrame(rows,columns=['scenario','product','year','kt']); S.to_csv('data/clean/scenarios_kt.csv',index=False)
tab=S[S.year.isin([2025,2026,2030,2035])].pivot_table(index=['scenario','product'],columns='year',values='kt').round(0)
); print(tab)
tot=S.groupby(['scenario','year']).kt.sum().unstack().round(0); print(tot)
# no-shock reference (PDP baseline growth only)
# supply security -----
ML={k:v for k,v in ML_PER_KT.items()}
dem26={p:S[(S.scenario=='Baseline')&(S['product']==p)&(S.year==2026)].kt.iloc[0] for p in PR}
dem25=base25
daily_ML={}
for p in ['Diesel (AGO)','Petrol (PMS)','Jet fuel*']:
    daily_ML[p]=dem25[p]*ML[p]/365
CAP={'Diesel (AGO)':755.04,'Petrol (PMS)':462.60,'Jet fuel*':231.84,'Kerosene (IK)':52.77} # PDP Table 5.7, gross
licensed tank capacity (ML), Mar 2025
rows=[]
for p in ['Diesel (AGO)','Petrol (PMS)','Jet fuel*']:
    d=daily_ML[p]; rows.append((p,d,CAP[p],CAP[p]/d,30*d,15*d))
SEC=pd.DataFrame(rows,columns=['product','daily_demand_ML(2025)','gross_capacity_ML','gross_days_if_full','ML_for_30d_
cover','ML_for_15d_operational']).round(1); print(SEC); SEC.to_csv('data/clean/security_capacity.csv',index=False)
# reported stock cover (days): official/press statements, 2026 (see report for sources)
REP=pd.DataFrame([['Petrol (PMS)',16,13,28],['Diesel (AGO)',19,16,23],['Jet
fuel*',49,46,None]],columns=['product','Treasury/MoE ~Apr 2 (days)','MoE report adj. ~Apr 2 (days)','EPRA 21 May
(days)']); REP.to_csv('data/clean/reported_cover.csv',index=False)
gap=[]
for p,lo,hi in [(['Petrol (PMS)',13,28),('Diesel (AGO)',16,23)]:
    d=daily_ML[p]; gap.append((p,lo,hi,30-hi,30-lo,(30-hi)*d,(30-lo)*d))
GAP=pd.DataFrame(gap,columns=['product','days_low','days_high','shortfall_days_min','shortfall_days_max','shortfall_ML
_min','shortfall_ML_max']).round(1); print(GAP); GAP.to_csv('data/clean/stock_gap.csv',index=False)
# LPG storage (PDP method: required storage = planned supply/12, planned supply = demand*(1+30/365))
cap_lpg=44.43; lp=[]
for sc in SC:
    for y in (2025,2026,2027,2028,2029,2030):
        d=S[(S.scenario==sc)&(S['product']=='LPG')&(S.year==y)].kt.iloc[0]; req=d*(1+30/365)/12;
        lp.append((sc,y,d,req,cap_lpg-req))
LPGS=pd.DataFrame(lp,columns=['scenario','year','demand_kt','required_storage_kt','headroom_kt']).round(1);
print(LPGS[LPGS.scenario=='Baseline']); LPGS.to_csv('data/clean/lpg_storage.csv',index=False)
# PDP backtest
bt=pd.DataFrame({'PDP_2025_fc_kt':{'Petrol (PMS)':1554.0,'Diesel
(AGO)':2297.6,'LPG':446.2,'Total':5425.5},'Actual_2025_kt':{'Petrol (PMS)':base25['Petrol (PMS)'],'Diesel
(AGO)':base25['Diesel (AGO)'],'LPG':base25['LPG'],'Total':base25['Total']})
bt['error_%']=(bt.Actual_2025_kt/bt.PDP_2025_fc_kt-1)*100; print(bt.round(1));
bt.round(1).to_csv('data/clean/pdp_backtest.csv')
# risk framework -----
cv={'Diesel (AGO)':7.3,'Petrol (PMS)':9.2,'LPG':10.7,'Jet fuel*':5.6}
fe={'Diesel (AGO)':2.75,'Petrol (PMS)':3.50,'LPG':3.09,'Jet fuel*':3.55} # best sMAPE from tournament
cover={'Diesel (AGO)':19,'Petrol (PMS)':16,'LPG':np.nan,'Jet fuel*':49} # lowest official-ish April figure;
LPG not reported
price={'Diesel (AGO)':30.8,'Petrol (PMS)':20.0,'LPG':np.nan,'Jet fuel*':np.nan} # Mar->Sep 2026 pump change; LPG/jet
not regulated/observed
infra={'Diesel (AGO)':1,'Petrol (PMS)':1,'LPG':4,'Jet fuel*':1} # PDP: adequate storage except LPG
(needs capacity by 2027)
conc={'Diesel (AGO)':4,'Petrol (PMS)':4,'LPG':4,'Jet fuel*':3} # reliance on Gulf-origin supply (G-
to-G for petrol/diesel); judgement
def rk(v,lo,hi,rev=False):
    x=np.clip((v-lo)/(hi-lo),0,1); return 1+4*(1-x if rev else x)
R=pd.DataFrame(index=list(cv))
R['Demand volatility']=[rk(cv[p],5,12) for p in R.index]
R['Forecast uncertainty']=[rk(fe[p],2.0,6.0) for p in R.index]
R['Stock cover']=[rk(cover[p],10,30,True) if not np.isnan(cover[p]) else 4.0 for p in R.index] # LPG: no published
cover -> 4 (data gap penalty)
R['Price shock']=[rk(price[p],0,30) if not np.isnan(price[p]) else 3.0 for p in R.index] # unobserved ->
neutral 3
R['Infrastructure gap']=[infra[p] for p in R.index]; R['Supply concentration']=[conc[p] for p in R.index]
Wl={'Stock cover':.25,'Price shock':.20,'Infrastructure gap':.15,'Demand volatility':.15,'Supply
concentration':.15,'Forecast uncertainty':.10}
Weq={k:1/6 for k in Wl}; Wst={'Stock cover':.4,'Price shock':.2,'Infrastructure gap':.1,'Demand volatility':.1,'Supply
concentration':.1,'Forecast uncertainty':.1}
for nm,wt in [(['Score (base weights)',Wl),(['Score (equal)',Weq),(['Score (stock-heavy)',Wst)]: R[nm]=sum(R[k]*v for k,v
in wt.items())

```

```

R['Rating']=pd.cut(R['Score (base weights)'],[0,2.4,3.2,5],labels=['LOW','MEDIUM','HIGH']);
print(R.round(2).to_string()); R.round(2).to_csv('data/clean/risk_scores.csv')
# figures
plt.rcParams.update({'font.size':9,'axes.spines.top':False,'axes.spines.right':False})
fig,ax=plt.subplots(figsize=(7.5,3.6)); col={'High-demand': '#2e7d32','Baseline': '#1f4e79','Low-demand': '#c0392b'}
for sc in SC: ax.plot(tot.columns,tot.loc[sc]/1000,'o-',ms=3,c=col[sc],label=sc)
ax.fill_between(tot.columns,tot.loc['Low-demand']/1000,tot.loc['High-demand']/1000,color='grey',alpha=.12)
ax.set_ylabel('Million tonnes'); ax.legend(frameon=False); ax.grid(alpha=.25); fig.tight_layout();
fig.savefig('figures/f6_scenarios.png',dpi=200); plt.close()
fig,ax=plt.subplots(figsize=(7.5,3.2));
for sc in SC:
ax.plot(LPGS[LPGS.scenario==sc].year,LPGS[LPGS.scenario==sc].required_storage_kt,'o-',c=col[sc],label=sc)
ax.axhline(cap_lpg,c='k',ls='--'); ax.text(2025.05,cap_lpg+.7,'Installed LPG storage Mar-2025: 44.43 kt (PDP)',fontSize=7)
ax.set_ylabel('Required LPG storage, kt'); ax.legend(frameon=False); ax.grid(alpha=.25); fig.tight_layout();
fig.savefig('figures/f7_lpg_storage.png',dpi=200); plt.close()

```

## A.7 code/lokichar\_model.py – South Lokichar cash-flow function

```

"""06 - South Lokichar illustrative economic model. ALL FISCAL AND COST INPUTS ARE ASSUMPTIONS (public headline facts:
~US$6bn capex commitment, 60-100 kbpd early production, ~560 mmbbl recoverable, first-oil target Dec 2026)."""
import numpy as np, pandas as pd, matplotlib; matplotlib.use('Agg'); import matplotlib.pyplot as plt
from scipy.optimize import brentq
BASE=dict(brent=70.0,diff=6.0,plateau=60.0,capex=6000.0,opex=12.0,tariff=9.0,disc=0.10,royalty=0.10,cr_cap=0.60,gov_pp
=0.50,decl=0.10,reserves=560.0,abex=0.03)
YEARS=np.arange(2026,2061)
RAMP={2026:0.0,2027:0.33,2028:0.66} # share of plateau (first oil Dec-2026 -> no 2026 volume)
CAPEX_PHASE={2026:.15,2027:.35,2028:.30,2029:.20}
def run(**kw):
    a=dict(BASE); a.update(kw); cum=0; rows=[]; plateau_end=2029+8 # plateau 2029-2037 (9 yrs)
    for y in YEARS:
        if y in RAMP: q=a['plateau']*RAMP[y]
        elif y<=plateau_end: q=a['plateau']
        else: q=rows[-1][1]*(1-a['decl'])
        vol=q*365/1000 # mmbbl
        if cum+vol>a['reserves']: vol=max(a['reserves']-cum,0)
        price=a['brent']-a['diff']; rev=vol*price; opx=vol*(a['opex']+a['tariff'])
        if vol>0 and rev<opx and y>plateau_end: vol=0; rev=opx=0 # economic limit
        cum+=vol; cap=a['capex']*CAPEX_PHASE.get(y,0)
        rows.append((y,q if vol>0 else 0,vol,rev,opx,cap));
    D=pd.DataFrame(rows,columns=['year','kbpd','mmbbl','revenue','opex','capex'])
    last=D[D.mmbbl>0].year.max(); D.loc[D.year==last+1,'capex']+=a['abex']*a['capex']; D=D[D.year<=last+1].copy()
    D['roy']=D.revenue*a['royalty']; D['pre_fiscal_cf']=D.revenue-D.opex-D.capex
    # PSC-style: cost recovery (opex+capex carried forward) capped at cr_cap of (revenue-royalty)
    pool=0; cr=[]; pp=[];
    for _,r in D.iterrows():
        pool+=r.opex+r.capex; avail=r.revenue-r.roy; rec=min(pool,a['cr_cap']*avail); pool-=rec; cr.append(rec);
    pp.append(max(avail-rec,0))
    D['cost_rec']=cr; D['profit_oil']=pp; D['gov_take']=D.roy+D.profit_oil*a['gov_pp']
    D['contractor_cf']=D.revenue-D.roy-D.profit_oil*a['gov_pp']-D.opex-D.capex
    t=D.year-2026; df=(1+a['disc'])**t
    out=dict(npv_project=(D.pre_fiscal_cf*df).sum(),npv_contractor=(D.contractor_cf*df).sum(),npv_gov=(D.gov_take*df).sum()
    ),gov_undisc=D.gov_take.sum(),
    cum_mmbbl=D.mmbbl.sum(),life_end=int(last),payback=None)
    def irr(cf):
        f=lambda r:(cf/(1+r)**t.values).sum()
        try: return brentq(f,-0.5,1.0)
        except: return np.nan
    out['irr_project']=irr(D.pre_fiscal_cf); out['irr_contractor']=irr(D.contractor_cf)
    c=D.pre_fiscal_cf.cumsum(); out['payback']=int(D.year[c>0].min()) if (c>0).any() else None
    tot=D.revenue.sum()-D.opex.sum()-D.capex.sum(); out['gov_share_pct']=D.gov_take.sum()/tot*100 if tot>0 else np.nan
    return D,out

```

## A.8 code/06\_project\_economics.py – South Lokichar scenarios, break-evens, tornado

```

"""06 - South Lokichar scenarios (model in lokichar_model.py)."""
import numpy as np, pandas as pd, matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as plt
from scipy.optimize import brentq
from lokichar_model import *
D,o=run(); print({k:(round(v,2) if isinstance(v,float) else v) for k,v in o.items()});
D.round(1).to_csv('data/clean/lokichar_cashflow_base.csv',index=False)
# break-evens
be_proj=brentq(lambda b:run(brent=b)[1]['npv_project'],20,150); be_ctr=brentq(lambda b:run(brent=b)[1]
['npv_contractor'],20,200); print('breakeven Brent project/contractor',round(be_proj,1),round(be_ctr,1))
# grid: price x production
prices=[50,60,70,80,90,100]; prod=[.8,.9,1.0,1.1,1.2]
G=pd.DataFrame({f'{int(f*100)}%':[run(brent=b,plateau=60*f)[1]['npv_project'] for b in prices] for f in
prod},index=prices).round(0); print(G); G.to_csv('data/clean/lokichar_grid_npv_project.csv')
Gc=pd.DataFrame({f'{int(f*100)}%':[run(brent=b,plateau=60*f)[1]['npv_gov'] for b in prices] for f in
prod},index=prices).round(0); Gc.to_csv('data/clean/lokichar_grid_npv_gov.csv')
# tornado on project NPV
base=o['npv_project']; T=[]
for nm,lo,hi,key in [['Oil price (Brent $55-$85)',dict(brent=55),dict(brent=85),'brent'],('Plateau output (-20%/+20%
of 60 kbpd)',dict(plateau=48),dict(plateau=72),'plateau'),
('CAPEX (+20%/-20%)',dict(capex=7200),dict(capex=4800),'capex'),('Opex+tariff (+20%/-
20%)',dict(opex=14.4,tariff=10.8),dict(opex=9.6,tariff=7.2),'opex'),('Discount rate
(12%/8%)',dict(disc=.12),dict(disc=.08),'disc'),('Price differential ($10/$2)',dict(diff=10),dict(diff=2),'diff')]:
    T.append((nm,run(**lo)[1]['npv_project']-base,run(**hi)[1]['npv_project']-base))
T=pd.DataFrame(T,columns=['driver','downside','upside']); T['swing']=T.upside.abs()+T.downside.abs();
T=T.sort_values('swing'); print(T.round(0)); T.round(0).to_csv('data/clean/lokichar_tornado.csv',index=False)
# scenario table (fiscal): Brent 50/70/90, plateau 60/100
S=[]
for b in (50,70,90):

```

```

for pl in (60,100):
    r=run(brent=b,plateau=pl);
    S.append((b,pl,r['npv_project'],r['irr_project'],r['npv_contractor'],r['irr_contractor'],r['npv_gov'],r['gov_undisc'],
    r['payback'],r['cum_mmbbl']))
    S=pd.DataFrame(S,columns=['brent','plateau_kbpd','NPV_project','IRR_project','NPV_contractor','IRR_contractor','NPV_gov',
    'gov_undisc','payback_year','cum_mmbbl']); print(S.round(2));
    S.round(3).to_csv('data/clean/lokichar_scenarios.csv',index=False)
    pd.Series({'be_proj':be_proj,'be_ctr':be_ctr}).to_csv('data/clean/lokichar_breakeven.csv')
    plt.rcParams.update({'font.size':9,'axes.spines.top':False,'axes.spines.right':False})
    fig,ax=plt.subplots(figsize=(7.5,3.3)); ax.barh(T.driver,T.downside,color='#c0392b');
    ax.barh(T.driver,T.upside,color='#2e7d32'); ax.axvline(0,c='k',lw=.7)
    ax.set_xlabel(f'Change in project NPV10 vs base (US$ m); base = {base:.0f}'); fig.tight_layout();
    fig.savefig('figures/f8_tornado.png',dpi=200); plt.close()
    fig,ax=plt.subplots(figsize=(7.5,3.0)); ax.bar(D.year,D.mmbbl,color='#1f4e79'); ax.set_ylabel('mmbbl / year');
    ax2=ax.twinx(); ax2.plot(D.year,D.pre_fiscal_cf.cumsum()/1000,c='#c0392b'); ax2.set_ylabel('Cumulative pre-fiscal cash
    flow, US$ bn'); fig.tight_layout(); fig.savefig('figures/f9_lokichar_profile.png',dpi=200); plt.close()

# alternative fiscal case reflecting press reports of 'enhanced cost recovery' (ASSUMED numbers): cap 80%, govt
profit-oil 35%
F=[]
for nm,kw in [(('Fiscal case A (cap 60%, gov profit oil 50%)',{}),('Fiscal case B (cap 80%, gov profit oil
35%)',dict(cr_cap=.80,gov_pp=.35))):
    for b in (60,70,80):
        r=run(brent=b,**kw); be=brentq(lambda x:run(brent=x,**kw)[1]['npv_contractor'],20,200)
        F.append((nm,b,r['npv_contractor'],r['irr_contractor'],r['npv_gov'],r['gov_undisc'],r['gov_share_pct'],be))
F=pd.DataFrame(F,columns=['fiscal_case','brent','NPV_contractor','IRR_contractor','NPV_gov','gov_undisc','gov_share_pc
t','contractor_breakeven_brent']); print(F.round(2));
F.round(3).to_csv('data/clean/lokichar_fiscal_cases.csv',index=False)

```

## A.9 code/09\_extended\_analysis.py – Extended analysis and Monte Carlo

```

"""09 - Extended analysis: product profiles, regional price spreads, landed-cost import bill, inventory carrying cost,
energy-transition arithmetic, scenario sensitivities, South Lokichar Monte Carlo, cooking-fuel cost comparison."""
import numpy as np, pandas as pd, matplotlib; matplotlib.use('Agg'); import matplotlib.pyplot as plt
from common import *; from lokichar_model import run, BASE
plt.rcParams.update({'font.size':9,'axes.spines.top':False,'axes.spines.right':False})
w=load_cons().dropna(); p=load_prices(); W=w
# ---- A. product profile tables: monthly YoY and 3m avg ----
yoy=(W.pct_change(12)*100).round(1); r3=W.rolling(3).mean().round(1)
yoy.to_csv('data/clean/monthly_yoy_table.csv'); r3.to_csv('data/clean/rolling3_table.csv')
prof=[]
for pr in ['Diesel (AGO)','Petrol (PMS)','LPG','Jet fuel*','Fuel oil/other*','Kerosene (IK)']:
    s=W[pr]; y=yoy[pr].dropna()
    prof.append((pr,s.max(),s.idxmax().strftime('%b-%y'),s.min(),s.idxmin().strftime('%b-
    %y'),s.mean(),s.std(),y.max(),y.min(),(y>0).mean()*100))
pd.DataFrame(prof,columns=['product','max_kt','max_month','min_kt','min_month','mean_kt','std_kt','best_yoy
%', 'worst_yoy%', 'share_months_growing%']).round(1).to_csv('data/clean/product_profiles.csv',index=False)
# ---- B. regional price spreads (Nairobi max, EPRA reviews as reported; KSh/L) ----
reg=pd.DataFrame([
    ['Nairobi',214.03,217.86,191.38],['Mombasa',210.87,214.58,188.09],['Kisumu',213.69,218.08,191.63],
    ['Nakuru',212.92,217.27,190.81],
    ],columns=['town','petrol_sep26','diesel_sep26','kerosene_sep26'])
reg2=pd.DataFrame([['Nairobi',214.25,242.92,152.78],['Mombasa',211.09,239.64,149.49],['Mandera',234.90,265.10,174.96],
    ['Nyeri',216.12,244.93,np.nan],['Embu',215.69,244.46,154.31],
    ['Moyale',229.10,258.86,np.nan]],columns=['town','petrol_may26','diesel_may26','kerosene_may26'])
reg2['petrol_vs_mombasa']=reg2.petrol_may26-211.09; reg2['diesel_vs_mombasa']=reg2.diesel_may26-239.64
reg2['petrol_vs_mombasa_%']=reg2.petrol_vs_mombasa/211.09*100
reg2.round(2).to_csv('data/clean/regional_spread_may26.csv',index=False);
reg.to_csv('data/clean/regional_prices_sep26.csv',index=False)
sep_extra={'Mandera petrol (Sep-26)':234.68,'Mombasa petrol (Sep-26)':210.87}
sp=sep_extra['Mandera petrol (Sep-26)']-sep_extra['Mombasa petrol (Sep-26)']
pd.Series({'mandera_minus_mombasa_petrol_sep26':sp,'pct':sp/210.87*100}).to_csv('data/clean/
regional_spread_sep26.csv')
# ---- C. price indices and cooking-fuel cost comparison ----
idx=(p/p.iloc[0]*100).round(1); idx.to_csv('data/clean/price_index_jan25_100.csv')
rows=[]
for lpg_eff,ch_eff in [(.55,.20),(.55,.25),(.55,.30),(.45,.30)]:
    for mon in ['2025-01-01','2026-03-01']:
        lp=p.loc[mon,'L.P.G']/13; ch=p.loc[mon,'Charcoal']
        rows.append((mon,[lpg_eff,ch_eff,lp/(46*lpg_eff),ch/(29*ch_eff)])
CK=pd.DataFrame(rows,columns=['month','LPG_stove_eff','charcoal_stove_eff','LPG_KSh_per_useful_MJ','charcoal_KSh_per_u
seful_MJ']); CK['LPG_over_charcoal']=CK.LPG_KSh_per_useful_MJ/CK.charcoal_KSh_per_useful_MJ;
CK.round(2).to_csv('data/clean/cooking_cost_per_MJ.csv',index=False)
# ---- D. landed-cost import-bill illustration (constant 2026 baseline volumes) ----
S=pd.read_csv('data/clean/scenarios_kt.csv'); b26=S[(S.scenario=='Baseline')&(S.year==2026)].set_index('product').kt
FX=129.2
bill=[]
for prod,usd_m3_a,usd_m3_b,lab in [('Petrol (PMS)',582.11,874.26,'Feb-26 vs Aug-26'),('Diesel
(AGO)',855.59,957.05,'Jul-26 vs Aug-26 (diesel Feb-26 landed cost not retrieved)']):
    ml=b26[prod]*ML_PER_KT[prod]; a=ml*usd_m3_a/1000; b=ml*usd_m3_b/1000 # ML * USD/m3 /1000 = USD m
    bill.append((prod,ml,usd_m3_a,usd_m3_b,a,b-a,(b-a)*FX/1000,lab))
BILL=pd.DataFrame(bill,columns=['product','volume_ML_2026base','landed_a_USD_m3','landed_b_USD_m3','bill_a_USDm','bill
_b_USDm','increase_USDm','increase_KShbn','comparison']).round(1);
BILL.to_csv('data/clean/landed_cost_bill.csv',index=False)
# ---- E. inventory value & carrying cost of strategic stock ----
dm={'Petrol (PMS)':(W.loc['2025','Petrol (PMS)'].sum()*ML_PER_KT['Petrol (PMS)']/365,874.26),'Diesel (AGO)':
(W.loc['2025','Diesel (AGO)'].sum()*ML_PER_KT['Diesel (AGO)']/365,957.05)}
inv=[]
for days in (15,30,60,90):
    val=sum(d*days*u/1000 for d,u in dm.values()); vol=sum(d*days for d,_ in dm.values())
    inv.append((days,val,val*FX/1000,val*0.10,val*0.10*FX/1000))
INV=pd.DataFrame(inv,columns=['days_of_cover','volume_ML','inventory_USDm','inventory_KShbn','carry_10pct_USDm_pa','ca
rry_10pct_KShbn_pa']).round(1); INV.to_csv('data/clean/inventory_cost.csv',index=False)
# ---- F. energy-transition arithmetic (petrol displaced, baseline 2035) ----

```

```

q35=[(S.scenario=='Baseline')&(S.year==2035)&(S['product']=='Petrol (PMS)')].kt.iloc[0];
d35=[(S.scenario=='Baseline')&(S.year==2035)&(S['product']=='Diesel (AGO)')].kt.iloc[0]
ET=[]
for share in (.02,.05,.10,.20):
    kt=q35*share; ml=kt*ML_PER_KT['Petrol (PMS)']; ET.append((share*100,kt,ml,ml*0.874,ml*0.874*FX/1000))
ET=pd.DataFrame(ET,columns=['petrol_displaced_%','kt','ml','import_saving_USDm_at_Aug26_landed','KShbn']).round(1);
ET.to_csv('data/clean/ev_displacement.csv',index=False)
# ---- G. scenario sensitivities (2030/2035 baseline total) ----
G={'Diesel (AGO)':3.52,'Petrol (PMS)':3.35,'LPG':7.26,'Jet fuel*':1.87,'Kerosene (IK)':-4.57}
EL={'Diesel (AGO)':(-0.185,-0.570),'Petrol (PMS)':(-0.096,-0.259),'LPG':(-0.15,-0.30),'Jet fuel*':(0,0),'Kerosene (IK)':(-0.2,-0.5)}
SH={'Diesel (AGO)':20,'Petrol (PMS)':15,'LPG':10,'Jet fuel*':0,'Kerosene (IK)':20}
base25=W.loc['2025'].sum()
def total(year,fade=.8,shock=1.,elm=1.,gadd=0.):
    tot=base25['Avgas']+0
    # fuel oil follows PDP path
    q=base25['Fuel oil/other*']
    for y in range(2026,year+1): q*=1+(4.68 if y<=2027 else -3.19)/100
    tot+=q
    for pr,g in G.items():
        sr,lr=EL[pr]; sr*=elm; lr*=elm; sh=np.log(1+SH[pr]*shock/100); q=base25[pr]
        for y in range(2026,year+1):
            gg=(g+gadd)/100*(1 if y<=2029 else fade); adj=sr*sh if y==2026 else ((lr-sr)*sh/2 if y in (2027,2028) else 0)
            q*=np.exp(np.log(1+gg)+adj)
            tot+=q
    return tot
SS=[('Baseline',total(2030),total(2035))]
for nm,kw in [ ('Fade 0.6 (slower post-2029)',dict(fade=.6)), ('Fade 1.0 (no fade)',dict(fade=1.0)), ('No 2026 price shock',dict(shock=0)), ('Shock x1.5',dict(shock=1.5)), ('Elasticities x0.5',dict(elm=.5)), ('Elasticities x2',dict(elm=2)), ('Underlying growth +2 pts/yr (2025 momentum)',dict(gadd=2.0)), ('Underlying growth -1 pt/yr',dict(gadd=-1.0))]:
    SS.append((nm,total(2030,**kw),total(2035,**kw)))
SS=pd.DataFrame(SS,columns=['case','total_2030_kt','total_2035_kt']);
SS['vs_baseline_2035_%']=(SS.total_2035_kt/SS.total_2030_kt-1)*100;
SS.round(1).to_csv('data/clean/scenario_sensitivity.csv',index=False); print(SS.round(1))
# ---- H. Monte Carlo on South Lokichar (project NPV10) ----
rng=np.random.default_rng(42); N=2000; out=[]
for _ in range(N):
    br=float(np.clip(rng.lognormal(np.log(70),0.22),35,140)); df=rng.uniform(3,10); pl=rng.triangular(40,60,100);
    cx=rng.triangular(5000,6000,8500)
    ox=rng.triangular(15,21,28); r=run(brent=br,diff=df,plateau=pl,capex=cx,opex=ox*12/21,tariff=ox*9/21)[1]
    out.append((br,df,pl,cx,ox,r['npv_project'],r['npv_contractor'],r['npv_gov']))
MC=pd.DataFrame(out,columns=['brent','diff','plateau','capex','opex_tariff','npv_project','npv_contractor','npv_gov'])
MC.round(1).to_csv('data/clean/lokichar_montecarlo.csv',index=False)
q=MC[['npv_project','npv_contractor','npv_gov']].quantile([.1,.5,.9]).round(0);
q.loc['P(NPV<0)']=(MC[['npv_project','npv_contractor','npv_gov']]<0).mean().round(3);
q.loc['mean']=MC[['npv_project','npv_contractor','npv_gov']].mean().round(0);
q.to_csv('data/clean/lokichar_mc_summary.csv'); print(q)
cor=MC.drop(columns=['npv_contractor','npv_gov']).corr()['npv_project'].drop('npv_project').round(2);
cor.to_csv('data/clean/lokichar_mc_corr.csv'); print(cor)
fig,ax=plt.subplots(figsize=(7.5,3.2)); ax.hist(MC.npv_project/1000,bins=45,color='#1f4e79',alpha=.85);
ax.axvline(0,c='k',lw=.8); ax.axvline(MC.npv_project.median()/1000,c='#c0392b',ls='--',label='median')
ax.set_xlabel('Project NPV10, US$ bn'); ax.set_ylabel('Draws'); ax.legend(frameon=False); fig.tight_layout();
fig.savefig('figures/f10_montecarlo.png',dpi=200); plt.close()
# ---- I. extra figures ----
R=pd.read_csv('data/clean/forecast_rolling_errors.csv'); tt=pd.read_csv('data/clean/forecast_tournament.csv');
best=tt.loc[tt.groupby('series').SMAPE.idxmin()].set_index('series').model
fig,ax=plt.subplots(1,2,figsize=(7.8,3.2))
for ax,s in zip(axs,['Diesel (AGO)','Total']):
    r=R[(R.series==s)&(R.model==best[s])&(R.h==1)]; dates=W.index[r.origin.values]
    ax.plot(W.index,W[s],c='#1f4e79',lw=1,label='actual'); ax.plot(dates,r.fc.values,'o',c='#c0392b',ms=4,label='1-month-ahead forecast'); ax.set_title(f'{s}: 1-step out-of-sample',fontsize=8.5); ax.grid(alpha=.25);
ax.tick_params(labelsize=7)
axs[0].legend(frameon=False,fontsize=7); fig.autofmt_xdate(); fig.tight_layout();
fig.savefig('figures/f11_backtest.png',dpi=200); plt.close()
fig,ax=plt.subplots(figsize=(7.5,3.2));
for c in ['petrol','diesel','kerosene','charcoal']: pass
pi=p.rename(columns={'Motor Gasoline Premium':'Petrol','Light Diesel Oil':'Diesel','Illuminating Kerosene':'Kerosene','L.P.G':'LPG (13kg)','Charcoal':'Charcoal'}); pi=pi/pi.iloc[0]*100
for c in pi.columns: ax.plot(pi.index,pi[c],label=c)
ax.set_ylabel('Index, Jan-2025 = 100'); ax.legend(ncol=5,frameon=False,fontsize=7); ax.grid(alpha=.25);
fig.tight_layout(); fig.savefig('figures/f12_price_index.png',dpi=200); plt.close()
fig,ax=plt.subplots(figsize=(7.5,3.4)); tab=W.loc['2024':'2025'].copy(); tab['m']=tab.index.month;
tab['y']=tab.index.year
for pr,c in [ ('Diesel (AGO)', '#1f4e79'), ('Petrol (PMS)', '#c0392b')]:
    for y,ls in [(2024,'-'),(2025,'-')]:
        t=tab[tab.y==y]; ax.plot(t.m,t[pr],ls=c,label=f'{pr} {y}')
ax.set_xticks(range(1,13)); ax.set_xlabel('Month'); ax.set_ylabel('kt'); ax.legend(ncol=2,frameon=False,fontsize=7);
ax.grid(alpha=.25); fig.tight_layout(); fig.savefig('figures/f13_seasonal_overlay.png',dpi=200); plt.close()
print(BILL.to_string()); print(INV.to_string()); print(ET.to_string()); print(CK.to_string())

```

## A.10 code/10\_build\_excel.py – Excel model builder

```

"""10 - Build Kenya_Petroleum_Economic_Model.xlsx with live formulas (change an assumption -> model updates)."""
import pandas as pd, numpy as np
from openpyxl import Workbook
from openpyxl.styles import Font, PatternFill, Alignment, Border, Side
from openpyxl.utils import get_column_letter as L
from common import *
wb=Workbook(); BLUE=Font(color='0000FF'); HB=Font(bold=True,color='FFFFFF'); HF=PatternFill('solid',fgColor='1F4E79');
INP=PatternFill('solid',fgColor='FF72CC'); BOLD=Font(bold=True)
def hdr(ws,row,vals,col=1):
    for i,v in enumerate(vals):

```

```

        c=ws.cell(row=row,column=col+i,value=v); c.font=HB; c.fill=HF;
c.alignment=Alignment(horizontal='center',wrap_text=True)
def inp(c): c.font=BLUE; c.fill=INP
def widths(ws,ws_w):
    for i,w in enumerate(ws_w): ws.column_dimensions[L(i+1)].width=w
# ----- README -----
ws=wb.active; ws.title='README'; widths(ws,[110])
for i,t in enumerate(['Kenya Petroleum Economic Model (companion to the Market Outlook 2026-2035)',',',
    'How it works: yellow cells with blue text are inputs (sheet Assumptions and the project inputs on
Project Economics). Everything else is a formula.',
    'Change the scenario selector (Assumptions!C3: 1 = Low-demand, 2 = Baseline, 3 = High-demand) and the Scenarios,
Supply_Security, LPG_Storage and Dashboard sheets update.',
    'Sheets: Assumptions | Demand_Data | Prices | Scenarios | Supply_Security | LPG_Storage | Project_Economics |
Sensitivity (static results from the Python pipeline) | Dashboard',
    'Data: LeadAfrik extract of KNBS Leading Economic Indicators, with the product columns RELABELLED after
reconciliation to EPRA PDP 2024 and KNBS 2025 (see Data Quality Report). Jet fuel* and Fuel oil/other* identities are
probable, not confirmed.',
    'Scenario growth rates: EPRA Petroleum Development Plan 2025-2029. Price shocks, 2030-35 fade, LPG/kerosene/jet
elasticities and all South Lokichar costs and fiscal terms are the author's ASSUMPTIONS.',
    'South Lokichar: illustrative public-information model; NOT official or confidential project economics.',
    'Sensitivity sheet values are static outputs of the Python model (they include break-even solves that Excel cannot do
without Goal Seek). The Project_Economics sheet reproduces the base case live.']):
    ws.cell(row=i+1,column=1,value=t).alignment=Alignment(wrap_text=True)
ws['A1'].font=Font(bold=True,size=14)
# ----- Demand_Data -----
W=load_cons(); wd=wb.create_sheet('Demand_Data'); hdr(wd,1,['Month','Year']+list(W.columns))
for i,(dt,r) in enumerate(W.iterrows()):
    wd.cell(row=i+2,column=1,value=dt.to_pydatetime()).number_format='mmm-yy'; wd.cell(row=i+2,column=2,value=dt.year)
    for j,v in enumerate(r.values): c=wd.cell(row=i+2,column=3+j,value=None if pd.isna(v) else float(v));
c.number_format='#,##0.0'
n=len(W)+1; widths(wd,[10,7]+[14]*9)
wd.cell(row=n+3,column=1,value='Annual totals (kt) - complete years only').font=BOLD
for k,y in enumerate((2024,2025)):
    wd.cell(row=n+4+k,column=1,value=y)
    for j in range(len(W.columns)): col=L(3+j); c=wd.cell(row=n+4+k,column=3+j,value=f'=SUMIFS({col}2:{col}{n},
$B$2:$B${n},$A{n+4+k})'); c.number_format='#,##0.0'
wd.cell(row=n+6,column=1,value='Growth 2025/24').font=BOLD
for j in range(len(W.columns)): col=L(3+j); c=wd.cell(row=n+6,column=3+j,value=f'={col}{n+5}/{col}{n+4}-1');
c.number_format='0.0%'
wd.cell(row=n+7,column=1,value='Share 2025').font=BOLD
for j in range(len(W.columns)): col=L(3+j); c=wd.cell(row=n+7,column=3+j,value=f'={col}{n+5}/$L(3+len(W.columns)-1)
{n+5}'); c.number_format='0.0%'
wd.freeze_panes='C2'; ANN25=n+5 # row of 2025 totals
cols={p:L(3+i) for i,p in enumerate(W.columns)}
# ----- Prices -----
pr=load_prices(); wp=wb.create_sheet('Prices'); hdr(wp,1,['Month']+list(pr.columns)); widths(wp,[10]+[22]*5)
for i,(dt,r) in enumerate(pr.iterrows()):
    wp.cell(row=i+2,column=1,value=dt.to_pydatetime()).number_format='mmm-yy'
    for j,v in enumerate(r.values): wp.cell(row=i+2,column=2+j,value=float(v)).number_format='#,##0.00'
m=len(pr)+1; wp.cell(row=m+2,column=1,value='Change Jan-25 to Mar-26').font=BOLD
for j in range(len(pr.columns)): c=wp.cell(row=m+2,column=2+j,value=f'={L(2+j)}{m}/L(2+j)}2-1');
c.number_format='0.0%'
wp.cell(row=m+3,column=1,value='CV').font=BOLD
for j in range(len(pr.columns)): c=wp.cell(row=m+3,column=2+j,value=f'=STDEV({L(2+j)}2:{L(2+j)}{m})/AVERAGE({L(2+j)}2:
{L(2+j)}{m})'); c.number_format='0.0%'
wp.cell(row=m+5,column=1,value='Note: national averages to Mar-2026. 2026 Nairobi maxima after the Gulf-war shock are
in the report (Diesel 242.92 in May-26; 217.86 in Sep-26).')
# ----- Assumptions -----
wa=wb.create_sheet('Assumptions',1); widths(wa,[44,14,14,14,50])
wa['A1']='ASSUMPTIONS (yellow = input)'; wa['A1'].font=Font(bold=True,size=13)
wa['A3']='Scenario selector (1=Low-demand, 2=Baseline, 3=High-demand)'; wa['B3']='Selected'; wa['C3']=2; inp(wa['C3'])
wa['D3']='=CHOOSE(C3,"Low-demand","Baseline","High-demand")'
prods=['Diesel (AGO)','Petrol (PMS)','LPG','Jet fuel*','Kerosene (IK)']
hdr(wa,5,['Underlying growth 2025-29, % per year','','Low (pessimistic)','Baseline','High (optimistic)','Selected /
source'])
G={ 'Diesel (AGO)':(2.74,3.52,4.28), 'Petrol (PMS)':(2.97,3.35,4.08), 'LPG':(6.34,7.26,8.82), 'Jet fuel*':
(1.70,1.87,2.17), 'Kerosene (IK)':(-5.29,-4.57,-3.86)}
for i,p in enumerate(prods):
    r=6+i; wa.cell(row=r,column=1,value=p)
    for j,v in enumerate(G[p]): c=wa.cell(row=r,column=3+j,value=v); inp(c)
    wa.cell(row=r,column=6,value=f'=INDEX(C{r}:E{r},$C$3)').number_format='0.00'
wa['A11']='Source: EPRA Petroleum Development Plan 2025-2029 (Section 4)'
hdr(wa,12,['2026 average pump-price rise vs 2025, %','','Low-demand','Baseline','High-demand','Selected'])
SH={ 'Diesel (AGO)':(30,20,10), 'Petrol (PMS)':(22,15,8), 'LPG':(20,10,5), 'Jet fuel*':(0,0,0), 'Kerosene (IK)':(30,20,10)}
for i,p in enumerate(prods):
    r=13+i; wa.cell(row=r,column=1,value=p)
    for j,v in enumerate(SH[p]): c=wa.cell(row=r,column=3+j,value=v); inp(c)
    wa.cell(row=r,column=6,value=f'=INDEX(C{r}:E{r},$C$3)')
wa['A18']='Author assumptions informed by Mar-Sep 2026 EPRA reviews (Nairobi diesel +30.8%, petrol +20.1% Mar-cycle to
Sep-cycle)'
hdr(wa,19,['Price elasticity of demand','','Short-run','Long-run','','Source'])
EL={ 'Diesel (AGO)':(-0.185,-0.570,'PDP-implied'), 'Petrol (PMS)':(-0.096,-0.259,'PDP-implied'), 'LPG':(-0.15,-
0.30,'Assumption'), 'Jet fuel*':(0,0,'Assumption'), 'Kerosene (IK)':(-0.2,-0.5,'Assumption')}
for i,p in enumerate(prods):
    r=20+i; wa.cell(row=r,column=1,value=p)
    for j in range(2): c=wa.cell(row=r,column=3+j,value=EL[p][j]); inp(c)
    wa.cell(row=r,column=6,value=EL[p][2])
oth=[('2030-35 funding fade factor (x 2025-29 rate)',0.8,'Assumption'),('Exchange rate, KSh per USD',129.2,'Assumption
(flat)'),('Fuel oil growth to 2027, % per year',4.68,'EPRA PDP'),('Fuel oil growth after 2027, % per year',-3.19,'EPRA
PDP'),
    ('Stock-cover standard, days',30,'EPRA PDP (15 operational + 15 strategic)'),('Installed LPG storage, kt (Mar-
2025)',44.43,'EPRA PDP'),('Additional LPG storage commissioned, kt (e.g. Lake Gas 10)',0,'Input; status unverified'),
    ('ML per kt: diesel',2608.2/2193.6,'PDP-implied'),('ML per kt: petrol',2044.11/1472.7,'PDP-implied'),('ML per kt:
jet',972.49/765.1,'PDP-implied'),('ML per kt: kerosene',47.17/37.1,'PDP-implied')]
for i,(a,b,s) in enumerate(oth):

```

```

r=26+i; wa.cell(row=r,column=1,value=a); c=wa.cell(row=r,column=3,value=b); inp(c);
wa.cell(row=r,column=6,value=s)
FADE,FX,F01,F02,COVER,LPGCAP,LPGADD,MLD,MLP,MLJ,MLK=[f'Assumptions!$C$26+i' for i in range(11)]
# ----- Scenarios -----
sc=wb.create_sheet('Scenarios'); widths(sc,[8]+[13]*9)
sc['A1']='Demand projection 2025-2035 (kt) - driven by the scenario selector'; sc['A1'].font=Font(bold=True,size=13);
sc['A2']='Assumptions!D3'
hdr(sc,4,['Year']+prods+'Fuel oil/other*','Avgas','Total','YoY total'])
pcol={'Diesel (AG0)':2,'Petrol (PMS)':3,'LPG':4,'Jet fuel*':5,'Kerosene (IK)':6}
for k,y in enumerate(range(2025,2036)):
    r=5+k; sc.cell(row=r,column=1,value=y)
    for i,p in enumerate(prods):
        col=pcol[p]; cl=L(col)
        if y==2025: f=f'Demand_Data!{cols[p]}{ANN25}'
        else:
            g=f'Assumptions!$F$6+i'; sh=f'Assumptions!$F$13+i'; sr=f'Assumptions!$C$20+i'; lr=f'Assumptions!
$D$20+i'
            f=f"={cl}{r-1}*EXP(LN(1+{g})*IF($A{r}<=2029,1,{FADE})/100)+IF($A{r}=2026,{sr}*LN(1+{sh}/
100),IF(OR($A{r}=2027,$A{r}=2028),({lr}-{sr})*LN(1+{sh}/100)/2,0))"
            sc.cell(row=r,column=col,value=f).number_format='#,##0'
            sc.cell(row=r,column=7,value=f'Demand_Data!{cols['Fuel oil/other*']}{ANN25}" if y==2025 else f="G{r-
1}*(1+IF($A{r}<=2027,{F01},{F02})/100)")).number_format='#,##0'
            sc.cell(row=r,column=8,value=f"Demand_Data!{cols['Avgas']}{ANN25}" if y==2025 else f="H{r-
1}")).number_format='#,##0.0'
            sc.cell(row=r,column=9,value=f"=SUM(B{r}:H{r})).number_format='#,##0'
            if y>2025: sc.cell(row=r,column=10,value=f"I{r}/I{r-1}").number_format='0.0%'
sc['A17']='CAGR 2025-35'; sc['I17']='(I15/I5)^(1/10)-1'; sc['I17'].number_format='0.00%'
sc['A19']='Check: 2025 total should equal KNBS 5.7 Mt'; sc['I19']='I5'; sc['I19'].number_format='#,##0'
# ----- Supply_Security -----
ss=wb.create_sheet('Supply_Security'); widths(ss,[34,16,16,16,16,16,16])
ss['A1']='Supply security: days of cover and storage'; ss['A1'].font=Font(bold=True,size=13)
hdr(ss,3,['Product','2025 demand (kt)','Daily demand (ML)','Gross licensed capacity (ML)','Days of demand if tanks
full','Stock for standard cover (ML)','Reported cover (days) - input'])
sp=[('Diesel (AG0)', 'B',MLD,755.04,19),('Petrol (PMS)', 'C',MLP,462.60,16),('Jet fuel*', 'E',MLJ,231.84,49)]
for i,(p,col,ml,cap,rep) in enumerate(sp):
    r=4+i; ss.cell(row=r,column=1,value=p); ss.cell(row=r,column=2,value=f'Scenarios!{col}5').number_format='#,##0'
    ss.cell(row=r,column=3,value=f"B{r}*{ml}/365").number_format='0.00'; c=ss.cell(row=r,column=4,value=cap); inp(c)
    ss.cell(row=r,column=5,value=f"D{r}/C{r}").number_format='0';
    ss.cell(row=r,column=6,value=f"C{r}*{COVER}").number_format='0'; c=ss.cell(row=r,column=7,value=rep); inp(c)
    hdr(ss,9,['Product','Reported cover (days)','Shortfall vs standard (days)','Shortfall (ML)','Status (author
thresholds)'])
    for i,(p,_,_,_) in enumerate(sp):
        r=10+i; ss.cell(row=r,column=1,value=p); ss.cell(row=r,column=2,value=f"G{4+i}")
        ss.cell(row=r,column=3,value=f"=MAX({COVER}-B{r},0)");
        ss.cell(row=r,column=4,value=f"C{r}*C{4+i}").number_format='0.0'
        ss.cell(row=r,column=5,value=f"=IF(B{r}>={COVER},"GREEN",IF(B{r}>=15,"AMBER","RED"))")
ss['A14']='Thresholds: GREEN >= standard (30 d); AMBER 15-29; RED < 15. Author framework, not official. Reported cover
inputs are from press reports of Ministry/Treasury statements (Apr-2026): petrol 16, diesel 19, jet 49.'
ss['A16']='Inventory value of cover (USD m) at Aug-2026 landed cost'; ss['A16'].font=BOLD
ss['A17']='Landed cost petrol (USD/m3)'; ss['B17']=874.26; inp(ss['B17']); ss['A18']='Landed cost diesel (USD/m3)';
ss['B18']=957.05; inp(ss['B18'])
ss['A19']='Days of cover to value'; ss['B19']=90; inp(ss['B19'])
ss['A20']='Inventory value (USD m)'; ss['B20']=B19*(C5*B17+C4*B18)/1000; ss['B20'].number_format='#,##0'
ss['A21']='Inventory value (KSh bn)'; ss['B21']=f=B20*FX/1000; ss['B21'].number_format='#,##0.0'
ss['A22']='Annual carrying cost at financing rate (USD m)'; ss['B22']=B20*C22; ss['C22']=0.10; inp(ss['C22']);
ss['B22'].number_format='#,##0.0'
# ----- LPG Storage -----
ls=wb.create_sheet('LPG_Storage'); widths(ls,[10,18,22,18,16])
ls['A1']='LPG storage requirement (PDP method: planned supply = demand x (1+cover/365); storage = supply / 12)';
ls['A1'].font=BOLD
hdr(ls,3,['Year','LPG demand (kt)','Required storage (kt)','Installed + new (kt)','Headroom (kt)'])
for k,y in enumerate(range(2025,2036)):
    r=4+k; ls.cell(row=r,column=1,value=y); ls.cell(row=r,column=2,value=f'Scenarios!D5+k').number_format='#,##0'
    ls.cell(row=r,column=3,value=f"B{r}*(1+{COVER}/365)/12").number_format='0.0';
    ls.cell(row=r,column=4,value=f"={LPGCAP}+{LPGADD}").number_format='0.0'
    ls.cell(row=r,column=5,value=f"D{r}-C{r}").number_format='+0.0;-0.0'
# ----- Project_Economics -----
pe=wb.create_sheet('Project_Economics'); widths(pe,[44,14]+[12]*17)
pe['A1']='South Lokichar - ILLUSTRATIVE scenario model (assumptions are the author's; not official project
economics)'; pe['A1'].font=Font(bold=True,size=12)
IN=[('Brent',US$/bbl,(real,flat),70),('Differential to Brent',US$/bbl,6),('Plateau production, kbpd',60),('Capex,
US$ m',6000),('Opex, US$/bbl',12),('Transport tariff, US$/bbl',9),('Discount rate',0.10),('Royalty, % of
revenue',0.10),('Cost-recovery cap, % of net revenue',0.60),('Government share of profit oil',0.50),('Annual decline
after plateau',0.10),('Recoverable reserves, mmbbl',560),('Abandonment, % of capex',0.03)]
names=['brent','diff','plat','capex','opex','tariff','disc','roy','cap','gpp','decl','res','abex']
REF={}
for i,((a,b),nm) in enumerate(zip(IN,names)):
    r=3+i; pe.cell(row=r,column=1,value=a); c=pe.cell(row=r,column=2,value=b); inp(c); REF[nm]=f'$B$3+r'
pe['A17']='Ramp (share of plateau): 2026 0% / 2027 33% / 2028 66%; plateau to 2037; capex phasing 15/35/30/20% in
2026-29'
hdr(pe,19,['Year','Ramp / phase','kbpd (raw)','kbpd (eff.)','mmbbl','Cum.
mmbbl','Revenue','Royalty','Opex+tariff','Capex','Pre-fiscal CF','Cost pool','Cost recovered','Profit oil','Govt
take','Contractor CF','Disc. factor'])
phase={2026:.15,2027:.35,2028:.30,2029:.20}; ramp={2026:0.0,2027:0.33,2028:0.66}
first=20
for k,y in enumerate(range(2026,2061)):
    r=first+k; pe.cell(row=r,column=1,value=y)
    pe.cell(row=r,column=2,value=ramp.get(y,1.0 if y<=2037 else None))
    pe.cell(row=r,column=3,value=f"={REF['plat']}*B{r}" if y<=2037 else f"D{r-1}*(1-
{REF['decl']})").number_format='0.0'
    pe.cell(row=r,column=4,value=f"=IF(E{r}>0,C{r},0)").number_format='0.0'
    prevcum='0' if k==0 else f"F{r-1}"
    pe.cell(row=r,column=5,value=f"=IF(AND(A{r}>2037,C{r}*365/1000*({REF['brent']}-
{REF['diff']})<C{r}*365/1000*({REF['opex']})+({REF['tariff']})),0,MIN(C{r}*365/1000,MAX({REF['res']}-
{prevcum},0)))").number_format='0.0'
    pe.cell(row=r,column=6,value=f"={prevcum}+E{r}").number_format='0.0'

```

```

pe.cell(row=r,column=7,value=f"E{r}*({REF['brent']}-{REF['diff']})").number_format='#,##0'
pe.cell(row=r,column=8,value=f"G{r}*({REF['roy']})").number_format='#,##0'
pe.cell(row=r,column=9,value=f"E{r}*({REF['opex']})+({REF['tariff']})").number_format='#,##0'
prevvol='0' if k==0 else f'E{r-1}'
pe.cell(row=r,column=10,value=f"={REF['capex']}*{phase.get(y,0)}+IF(AND({prevvol}>0,E{r}=0),
{REF['abex']}*{REF['capex']},0)").number_format='#,##0'
pe.cell(row=r,column=11,value=f"G{r}-I{r}-J{r}").number_format='#,##0'
prevpool='0' if k==0 else f'L{r-1}'
pe.cell(row=r,column=13,value=f"=MIN({prevpool}+I{r}+J{r},{REF['cap']}*(G{r}-H{r}))").number_format='#,##0'
pe.cell(row=r,column=12,value=f"={prevpool}+I{r}+J{r}-M{r}").number_format='#,##0'
pe.cell(row=r,column=14,value=f"=MAX(G{r}-H{r}-M{r},0)").number_format='#,##0'
pe.cell(row=r,column=15,value=f"H{r}+N{r}*({REF['gpp']})").number_format='#,##0'
pe.cell(row=r,column=16,value=f"G{r}-H{r}-N{r}*({REF['gpp']}-I{r}-J{r})").number_format='#,##0'
pe.cell(row=r,column=17,value=f"=(1+{REF['disc']})^(A{r}-2026)").number_format='0.000'
last=first+34
pe['A57']='RESULTS'; pe['A57'].font=BOLD
res=[('Project NPV (pre-fiscal), US$ m',f'=SUMPRODUCT(K{first}:K{last},Q{first}:Q{last})', '#,##0'), ('Project
IRR',f'=IRR(K{first}:K{last},0.1)', '0.0%'), ('Government NPV, US$
m',f'=SUMPRODUCT(O{first}:O{last},Q{first}:Q{last})', '#,##0'), ('Government take, undiscounted, US$
m',f'=SUM(O{first}:O{last})', '#,##0'), ('Contractor NPV, US$
m',f'=SUMPRODUCT(P{first}:P{last},Q{first}:Q{last})', '#,##0'), ('Contractor
IRR',f'=IRR(P{first}:P{last},0.05)', '0.0%'), ('Cumulative production, mmbbl',f'=SUM(E{first}:E{last})', '#,##0'),
('Check: python base-case project NPV = 1,551; contractor = -1,260',None,None)]
for i,(a,f,nf) in enumerate(res):
    pe.cell(row=58+i,column=1,value=a)
    if f: c=pe.cell(row=58+i,column=2,value=f); c.number_format=nf
# ----- Sensitivity (static from python) -----
se=wb.create_sheet('Sensitivity'); widths(se,[30,14,14,14,14,14])
se['A1']='Static outputs from the Python pipeline (data/clean/*.csv)'; se['A1'].font=BOLD
g=pd.read_csv('data/clean/lokichar_grid_npv_project.csv',index_col=0); hdr(se,3,['Project NPV10, US$ m: Brent \
plateau']+['{c} of 60 kbpd' for c in g.columns])
for i,idx in enumerate(g.index):
    se.cell(row=4+i,column=1,value=f'US${idx}')
    for j,v in enumerate(g.loc[idx]): se.cell(row=4+i,column=2+j,value=float(v)).number_format='#,##0'
t=pd.read_csv('data/clean/lokichar_tornado.csv'); hdr(se,12,['Tornado driver','Downside (US$ m)','Upside (US$ m)'])
for i,r in enumerate(t.itertuples()): se.cell(row=13+i,column=1,value=r.driver);
se.cell(row=13+i,column=2,value=float(r.downside)).number_format='#,##0';
se.cell(row=13+i,column=3,value=float(r.upside)).number_format='#,##0'
f=pd.read_csv('data/clean/lokichar_fiscal_cases.csv'); hdr(se,21,list(f.columns))
for i,r in enumerate(f.itertuples(index=False)):
    for j,v in enumerate(r): se.cell(row=22+i,column=1+j,value=(float(v) if not isinstance(v,str) else v))
ss=pd.read_csv('data/clean/scenario_sensitivity.csv'); hdr(se,30,list(ss.columns))
for i,r in enumerate(ss.itertuples(index=False)):
    for j,v in enumerate(r): se.cell(row=31+i,column=1+j,value=(float(v) if not isinstance(v,str) else v))
# ----- Dashboard -----
db=wb.create_sheet('Dashboard',1); widths(db,[46,18,18,18])
db['A1']='KENYA PETROLEUM - DASHBOARD'; db['A1'].font=Font(bold=True,size=14); db['A2']='Assumptions!D3'
rows=[('Total demand 2025 (kt)', '=Scenarios!I5', '#,##0'), ('Total demand 2026 (kt)', '=Scenarios!I6', '#,##0'), ('Total
demand 2030 (kt)', '=Scenarios!I10', '#,##0'), ('Total demand 2035 (kt)', '=Scenarios!I15', '#,##0'), ('CAGR 2025-
35', '=Scenarios!I17', '0.00%'),
('Diesel 2030 (kt)', '=Scenarios!B10', '#,##0'), ('Petrol 2030 (kt)', '=Scenarios!C10', '#,##0'), ('LPG 2030
(kt)', '=Scenarios!D10', '#,##0'), ('LPG storage headroom 2030 (kt)', '=LPG_Storage!E9', '+0.0;-0.0'), ('LPG storage
headroom 2026 (kt)', '=LPG_Storage!E5', '+0.0;-0.0'),
('Diesel shortfall vs standard (ML)', '=Supply_Security!D10', '#,##0'), ('Petrol shortfall vs standard
(ML)', '=Supply_Security!D11', '#,##0'), ('Project NPV10, US$ m', '=Project_Economics!B58', '#,##0'), ('Project
IRR', '=Project_Economics!B59', '0.0%'), ('Government NPV10, US$ m', '=Project_Economics!B60', '#,##0'), ('Contractor NPV10,
US$ m', '=Project_Economics!B62', '#,##0')]
for i,(a,f,nf) in enumerate(rows): db.cell(row=4+i,column=1,value=a); c=db.cell(row=4+i,column=2,value=f_);
c.number_format=nf
wb.save('out/Kenya_Petroleum_Economic_Model.xlsx'); print('saved')

```

## Appendix B. Output files

- consumption\_relabelled\_kt.csv
- cooking\_cost\_per\_MJ.csv
- demand\_summary.csv
- ev\_displacement.csv
- forecast\_12m.csv
- forecast\_rolling\_errors.csv
- forecast\_tournament.csv
- inventory\_cost.csv
- landed\_cost\_bill.csv
- lokichar\_breakeven.csv
- lokichar\_cashflow\_base.csv
- lokichar\_fiscal\_cases.csv
- lokichar\_grid\_npv\_gov.csv
- lokichar\_grid\_npv\_project.csv
- lokichar\_mc\_corr.csv
- lokichar\_mc\_summary.csv

- lokichar\_montecarlo.csv
- lokichar\_scenarios.csv
- lokichar\_tornado.csv
- lpg\_storage.csv
- monthly\_yoy\_table.csv
- own\_elasticity.csv
- passthrough.csv
- pdp\_backtest.csv
- pdp\_implied\_elasticity.csv
- price\_index\_jan25\_100.csv
- price\_stats.csv
- product\_profiles.csv
- pump\_path\_2026.csv
- reconciliation.csv
- regional\_prices\_sep26.csv
- regional\_spread\_may26.csv
- regional\_spread\_sep26.csv
- reported\_cover.csv
- risk\_scores.csv
- rolling3\_table.csv
- scenario\_sensitivity.csv
- scenarios\_kt.csv
- seasonal\_index.csv
- security\_capacity.csv
- stock\_gap.csv
- yoy.csv